

AroundYou

Sviluppo (Deliverable 3)

Progetto del corso di Ingegneria del Software presso l'Università di Trento, a.a. 2024-25.

Gruppo T31, membri: Albiero Alessandro, Bignotti Francesco, Dalla Vecchia Daniele

Indice

1. Scopo del documento.....	3
User Stories	4
<i>User story 1: La ricerca delle attività ottimizzata per topic o parola.....</i>	<i>4</i>
Criteri di accettazione	4
Tasks.....	4
Difficoltà.....	5
Tempo	5
<i>User story 2: L'iscrizione dell'utente al servizio.....</i>	<i>5</i>
Criteri di accettazione	5
Tasks.....	5
Difficoltà.....	6
Tempo	6
<i>User story 3: Autenticazione utente (login).....</i>	<i>6</i>
Criteri di accettazione	6
Tasks.....	6
Difficoltà.....	6
Tempo	6
<i>User story 4: Creazione di una nuova attività</i>	<i>6</i>
Criteri di accettazione	7
Tasks.....	7
Difficoltà.....	7
Tempo	7
<i>User story 5: Iscrizione ad una attività.....</i>	<i>7</i>
Criteri di accettazione	7
Tasks.....	8
Difficoltà.....	8
Tempo	8
<i>User story 6: Segnalazione di una attività</i>	<i>8</i>
Criteri di accettazione	8

Tasks.....	8
Difficoltà.....	9
Tempo	9
<i>User story 7: Visualizzazioni attività create (dashboard user e admin).....</i>	<i>9</i>
Criteri di accettazione	9
Tasks.....	9
Difficoltà.....	10
Tempo	10
<i>User story 8: Modifica attività create (dashboard user e admin)</i>	<i>10</i>
Criteri di accettazione	10
Tasks.....	10
Difficoltà.....	10
Tempo	10
<i>User story 9: Elimina attività create (dashboard user e admin).....</i>	<i>11</i>
Criteri di accettazione	11
Tasks.....	11
Difficoltà.....	11
Tempo	11
<i>User story 10: Visualizzazione attività a cui partecipo (dashboard user).....</i>	<i>11</i>
Criteri di accettazione	12
Tasks.....	12
Difficoltà.....	12
Tempo	12
<i>User story 11: modifica username email (dashboard user).....</i>	<i>12</i>
Criteri di accettazione	12
Tasks.....	13
Difficoltà.....	13
Tempo	13
<i>User story 12: elimina attività segnalate (dashboard admin)</i>	<i>13</i>
Criteri di accettazione	13
Tasks.....	14
Difficoltà.....	14
Tempo	14
<i>User story 13: Promuovi user ad admin (dashboard admin).....</i>	<i>14</i>
Criteri di accettazione	14
Tasks.....	15
Difficoltà.....	15
Tempo	15
<i>User story 14: Elimina user sospetto (dashboard admin)</i>	<i>15</i>
Criteri di accettazione	15
Tasks.....	16
Difficoltà.....	16

Tempo	16
2. User Flows	16
3. Web Apis	19
4. Implementazione	43
4.1 Repository Organization.....	44
4.2 Branching strategy e organizzazione del lavoro.....	44
4.3 Dependencies	46
4.4 Database	46
5. Testing	49
6. Front-end	73
6.1 Pagine di AroundYou.....	73
6.1.1 Landing page.....	75
6.1.2 Home page - mappa e attività.....	75
6.1.3 Pagina creazione nuova attività.....	76
6.1.4 Pagina di registrazione utente.....	77
6.1.5 Pagina di login.....	78
6.1.6 Dashboard utente semplice.....	78
6.1.7 Dashboard admin.....	81
7. Deployment.....	81

1. Scopo del documento

Il presente documento ha l'obiettivo di riportare tutte le informazioni necessarie per lo sviluppo della applicazione AroundYou, delineando in modo chiaro e strutturato le varie parti del progetto.

All'inizio viene presentata una descrizione dettagliata delle "User Stories", in modo da definire in modo specifico i requisiti funzionali dell'applicazione da parte dell'utente.

Successivamente verranno affrontate gli "User flow", diagrammi che descrivono il percorso di un generico utente compie sull'app per raggiungere un determinato obiettivo, e servono a garantire un'esperienza utente ottimale.

Il documento prosegue con la presentazione delle “Web API” del servizio web, e ne specifica le funzionalità, gli endpoint disponibili e il loro utilizzo.

Ci si sofferma poi sul “modello dati”, osservando le entità principali e le relazioni tra loro instaurate, in modo da fornire una solida base per la gestione delle informazioni all’interno dell’applicazione. Seguendo il procedimento di sviluppo del progetto, anche il documento si concentra sulla fase di “testing”, raccontando le metodologie, gli strumenti e le pratiche migliori per garantire la qualità del sito, oltre a dedicare dello spazio per parlare dello sviluppo del “front-end”, analizzando i procedimenti, le librerie utilizzate e la distribuzione del prodotto finito.

User Stories

User story 1: La ricerca delle attività ottimizzata per topic o parola

Come utente o operatore voglio poter cercare attività filtrando per nome, topic, luogo, e creatore in modo da individuare e analizzare quelle più affini alle mie necessità, in tempistiche brevi, senza dover sfogliare intere pagine.

Criteri di accettazione

- Deve esserci una barra di ricerca nella “home page” per inserire i parametri che permettono di filtrare le attività
- Devono esserci dei filtri istantanei in modo da ridurre le tempistiche
- Il sistema restituisce in tempo reale una lista aggiornata di attività che rispettano i criteri, valutati secondo una determinata priorità e i marker sulla mappa devono seguire gli aggiornamenti in tempo reale
- Se non ci sono attività che rispettano i criteri inseriti dall’utente, il sistema deve ritornare una pagina vuota
- Ogni risultato deve mostrare i campi principali di un’attività: Nome, Topics, Luogo, Data, Creatore, Posti disponibili

Tasks

- Creare il campo di ricerca nell’UI
- Sviluppo di topics (bottoni) istantanei e barra di ricerca che prende in input stringhe inserite dall’utente
- Implementare la logica di ricerca
- Progettare Database e Backend, e quindi implementare l’endpoint API per ricevere le richieste di filtraggio
- Costruire un algoritmo di ricerca in modo da poter richiedere al database, per mezzo di specifiche query, di restituire attività che seguono i criteri inseriti

- Testare i casi limite della ricerca per trovare errori e per analizzare le tempistiche

Difficoltà



Tempo

3 settimane.

User story 2: L'iscrizione dell'utente al servizio

Come utente non autenticato voglio poter creare un account utente in modo da poter accedere a tutte le funzionalità del sistema, incluse la creazione e gestione delle mie attività.

Criteri di accettazione

- L'utente non autenticato può iscriversi al sistema, inserendo nome, cognome, username (unico), email (unico), password (deve rispettare dei criteri di sicurezza)
- Il sistema ottiene le informazioni dell'utente che si sta iscrivendo e controlla la validità e l'unicità dello username, dell' email oltre a controllare se la password rispetta i criteri imposti
- Il sistema deve dare un riscontro all'utente se la registrazione è stata effettuata correttamente o se si sono riscontrati problemi durante il processo.

Tasks

- Creare la schermata di registrazione contenente il form da compilare inserendo i propri dati personali
- Aggiungere i messaggi di errore in tempo reale per guidare l'utente nella compilazione corretta del modulo
- Creare avvisi che permettono la corretta comprensione dell'avvenuta registrazione o del fallimento di quest'ultima
- Implementazione endpoint API per la registrazione utente
- Implementare i meccanismi di controllo di validità dei dati
 - Controllo unicità username e indirizzo email
 - controllo che vengano soddisfatti i requisiti minimi (lunghezza e caratteri password)
- Salvare i dati dell'utente nel database solo se i controlli sono stati superati
- Implementare una corretta criptazione hash utilizzando la libreria "bcrypt" per la memorizzazione delle password
- Testare la validazione dei dati nel form (inserimento mail errata, username già utilizzato, password troppo debole, email già presente)
- Provare la registrazione con dati duplicati e controllare i risultati ritornati dal sistema

Difficoltà



Tempo

1 settimana

User story 3: Autenticazione utente (login)

Come utente non autenticato voglio poter accedere al mio account con le mie credenziali locali ovvero nome utente e password

Criteri di accettazione

- L'utente non autenticato può accedere al sistema utilizzando nome utente e password
- Dopo un login riuscito, il sistema reindirizza l'utente alla dashboard corrispondente al suo ruolo
- Se il login fallisce, il sistema fornisce messaggi d'errore adeguati

Tasks

- Integrare correttamente il form di login nella pagina apposita oltre al pulsante registrati che ti porta alla registrazione nel caso l'utente non sia ancora registrato
- Gestire correttamente il routing dopo il login per entrambi i ruoli (utente e operatore), ovvero controllare che dopo l'autenticazione l'utente venga reindirizzato alla dashboard corretta in base al ruolo
- Implementare messaggi di errori specifici per problemi relativi al login
- Creare l'endpoint API per l'autenticazione con nome utente e password
- Validare le credenziali dell'utente e restituire un token di autenticazione che viene salvato nello spazio locale dell'utente
- Testare la modalità di accesso, osservando come il sistema risponde nel caso la password non combaccia con il nome utente o se si presentano altri tipi di errore

Difficoltà



Tempo

2 settimane

User story 4: Creazione di una nuova attività

Come utente autenticato voglio poter creare nuove attività specificando i dettagli obbligatori richiesti

Criteri di accettazione

- L'utente autenticato può creare attività inserendo necessariamente i campi obbligatori richiesti dal sistema
- Il sistema verifica la validità dei campi inseriti e, se corretti, porta a termine la creazione dell'attività
- Se mancano campi obbligatori o i dati non sono validi, il sistema fornisce un messaggio di errore chiaro
- L'attività viene salvata nel database e diventa disponibile nel sistema

Tasks

- Definire la struttura di un'attività definendo i campi obbligatori
- Creare una schermata di inserimento di nuove attività formata da:
 - form con campi per l'inserimento dei dati
 - tasto per poter inviare i dati del form
 - messaggi di errore in caso di campi mancanti o errati
- Modificare il backend per supportare la ricezione di nuove attività
- Creare un endpoint API per ricevere i dati dell'attività e salvarli nel database
- Aggiornare il database ad ogni nuova attività creata
- testare la validazione dei dati nel form, provando i casi che creano errore
- verificare che il backend prenda in modo corretto i dati e li salvi nel DB
- Verificare che i feedback restituiti dal sistema siano chiari e corretti in base alle situazioni

Difficoltà



Tempo

2-3 giorni

User story 5: Iscrizione ad una attività

Come utente autenticato voglio poter iscrivermi ad un'attività e avere il posto assicurato, in modo da non rischiare di rimanere escluso

Criteri di accettazione

- L'utente autenticato può iscriversi ad un'attività se sono ancora disponibili posti
- Il sistema verifica e impedisce a un utente di registrarsi più d'una volta
- Se l'iscrizione avviene con successo, l'attività viene aggiunta alla lista delle attività partecipate dell'utente
- Se i posti sono terminati e l'utente prova a iscriversi il sistema restituisce un messaggio di errore

- Il sistema impedisce agli utenti non autenticati di iscriversi

Tasks

- Modificare la struttura dell'utente per includere la lista di attività a cui partecipa
- Creare un endpoint API per gestire le iscrizioni alle attività
- Implementare il controllo della disponibilità di posti prima di confermare l'iscrizione
- Implementare il limite di una sola iscrizione ad una attività per utente
- aggiornare il database in modo che possa accettare e salvare l'iscrizione dell'utente autenticato all'attività
- Aggiungere il pulsante di iscrizione per ogni attività
- Implementare messaggi di errore chiari in caso di problemi (es. posti esauriti, iscrizione già effettuata).
- Aggiornare l'interfaccia utente dopo l'iscrizione, mostrando un messaggio di conferma.
- Verificare che un utente non possa iscriversi più di una volta alla stessa attività
- verificare che non avvenga l'iscrizione se sono esauriti i posti

Difficoltà



Tempo

1 settimana

User story 6: Segnalazione di una attività

Come utente autenticato voglio poter segnalare al comune possibili attività sospette che ritengo non vere o pericolose, così che vengano esaminate e se necessario rimosse

Criteri di accettazione

- L'utente autenticato può segnalare un'attività qualsiasi per mezzo di un tasto
- Il sistema controlla che l'utente non abbia già segnalato quella stessa attività
- Se la segnalazione avviene con successo si incrementa il numero totale di segnalazioni dell'attività e viene salvato l'id del segnalatore in una lista dell'attività
- Il sistema impedisce agli utenti non autenticati di inviare segnalazioni

Tasks

- Modificare lo schema dell'attività in modo da poter incrementare il numero di segnalazioni e per poter aggiungere l'id del segnalatore ad una lista di IdSegnalatori dell'attività
- Creare un endpoint API per ricevere le segnalazioni e verificarne la validità

- Implementare il controllo che ogni utente può segnalare ogni attività solamente una volta, e non si presentino segnalazioni duplicate
- Incrementare il numero di segnalazioni dell'attività ogni volta che viene segnalata da un nuovo utente
- Aggiungere l'Id di chi ha segnalato ad una lista dell'attività
- Creare un pulsante segnala per ogni attività
- Mostrare un messaggio di conferma nel caso la segnalazione sia avvenuta con successo, o un alert di errore nel caso ci sia stato un problema nella segnalazione (già segnalata oppure problema al server)
- Testare il corretto funzionamento del pulsante segnala
- Testare la corretta registrazione delle segnalazioni nel database
- Verificare che effettivamente un utente non può segnalare più di una volta una attività
- Assicurarsi che gli utenti non registrati non possono segnalare le attività

Difficoltà



Tempo

1 settimana

User story 7: Visualizzazioni attività create (dashboard user e admin)

Come utente autenticato voglio poter vedere nella mia area personale (dashboard) l'elenco delle attività da me create così che le posso monitorare e gestire facilmente

Criteri di accettazione

- L'utente autenticato può accedere alla dashboard e vedere le sue attività create con i dettagli come titolo, numero di partecipanti e data
- Il sistema deve aggiornare dinamicamente la lista quanto vengono create attività nuove, modificate o eliminate
- se un utente o un admin non ha attività visibili, la parte di pagine con la lista restituisce una riga bianca

Tasks

- Creare un endpoint api che recupera tutte le attività create da un utente specifico
- Aggiungere una sezione nella dashboard utente e admin con la lista di attività create
- aggiungere di bottoni per poter modificare ogni attività o eliminarla
- testare il recupero corretto delle attività del database per ogni utente
- testare l'aggiornamento dinamico della lista di attività nel caso una di esse venga eliminata, modificata o si creino nuove attività

Difficoltà



Tempo

2 giorni

User story 8: Modifica attività create (dashboard user e admin)

Come utente autenticato, voglio poter modificare le attività da me creato, nel caso si siano presentati imprevisti o si debbano modificare alcuni dati come la location o la data, così che i partecipanti possono essere informati dei cambiamenti

Criteri di accettazione

- L'utente autenticato può modificare l'attività grazie a un modulo che si apre dopo aver premuto il tasto "modifica" nella sua parte di dashboard con tutte le attività create
- Il sistema verifica che le modifiche rispettino i vincoli sui dati
- Se la modifica avviene con successo l'attività si aggiorna dinamicamente senza bisogno di ricaricare la pagina
- Se si presenta un problema l'attività non viene modificata e il sistema notifica l'errore

Tasks

- Creare un endpoint API per ricevere le modifiche apportate alla attività
- Implementare i controlli di validazione per garantire che i nuovi dati inseriti siano corretti
- Aggiornare il database con le nuove informazioni
- Aggiungere un pulsante "modifica" per ogni attività creata presente nella dashboard
- Creare il modulo che si palesa solamente una volta premuto il pulsante "modifica" e che permette di modificare i campi dell'attività
- Implementare l'aggiornamento dinamico della lista di attività create dopo una modifica
- testare il pulsante per modificare, il modulo con i campi modificabile e il salvataggio di questi ultimi
- testare l'aggiornamento dinamico della attività dopo una modifica
- simulare un errore e osservare la risposta del sistema

Difficoltà



Tempo

1 settimana

User story 9: Elimina attività create (dashboard user e admin)

Come utente autenticato voglio poter eliminare le mie attività create, nel caso non si possa più tenere o ci siano problemi con l'organizzazione, così che non risulti più visibile altri utenti, in primis a coloro che avrebbero partecipato

Criteri di accettazione

- L'utente autenticato può eliminare le proprie attività create per mezzo di un pulsante elimina
- Il sistema prende la richiesta dell'utente e elimina l'attività dal database, se l'eliminazione avviene con successo
- Se l'eliminazione non riesce per un errore, il sistema mostra un messaggio di errore
- il sistema aggiorna in modo dinamico la lista di attività create

Tasks

- Creare un endpoint API, per gestire l'eliminazione di un'attività dal database
- Implementare i controlli di autorizzazione che solo l'utente proprietario o un admin può eliminare un'attività creata
- Controllare che l'eliminazione rimuova tutti i dati collegati
- Aggiungere un pulsante "elimina" accanto a ogni attività creata nella dashboard
- Aggiornare la dashboard dinamicamente dopo l'evoluzione di un'attività
- Mostrare i messaggi di errore nel caso non avvenisse in modo corretto l'operazione di eliminazione
- Verificare che un utente possa eliminare le proprie attività
- Simulare un'eliminazione e controllare la risposta del server
- Controllare che la lista di attività si sia aggiornata dopo un'eliminazione
- Simulare un errore per osservare il messaggio di errore

Difficoltà



Tempo

1 giorno

User story 10: Visualizzazione attività a cui partecipo (dashboard user)

Come utente autenticato voglio poter vedere le attività a cui partecipo e le informazioni di esse in modo da controllare se ci sono state o meno delle modifiche

Criteri di accettazione

- L'utente autenticato può accedere alla parte "saved" della dashboard e visionare le attività a cui partecipa
- Il sistema restituisce le attività a cui ogni utente partecipa, solo a quelle che partecipa
- Ogni attività mostra le informazioni essenziali in modo che l'utente rimanga aggiornato sulle modifiche
- Se un'attività viene cancellata, anche dalla dashboard dell'utente partecipante viene eliminata
- Se l'utente non è iscritto a nessuna attività, il sistema restituisce la lista vuota (una sola riga bianca)

Tasks

- Creare un endpoint API per restituire all'utente tutte le attività a cui si è iscritto
- Implementare un sistema che la lista viene aggiornata dinamicamente se avvengono delle modifiche o cancellazioni
- Assicurarsi che solo l'utente autenticato possa vedere le proprie iscrizioni
- Creare una sezione nella dashboard utente denominata "saved" con la lista delle attività a cui partecipa
- mostrare un'interfaccia chiara e semplice, con i dettagli sulle attività
- Testare che l'utente può vedere solamente le attività a cui partecipa
- testare che le modifiche alle attività si riflettano correttamente nella dashboard

Difficoltà



Tempo

2-3 giorni

User story 11: modifica username email (dashboard user)

Come utente autenticato voglio poter modificare il mio username o la mia email, nel caso avessi cambiato email o il mio nome utente non mi piaccia più

Criteri di accettazione

- L'utente autenticato può entrare nella parte di dashboard denominata "settings" e può modificare lo username e il nome utente
- Il sistema controlla che lo username nuovo sia unico, e che la mail non sia associata a un altro account
- Se il nome o la mail sono già in uso il sistema informa l'utente di ciò
- Il sistema impedisce agli utenti di modificare i propri dati se non sono unici

Tasks

- Creare un endpoint API per ricevere le nuove modifiche e verificarne la validità
- Implementare il controllo che il nome utente e la email siano uniche, e mai utilizzate
- Implementare i messaggi di errore nel caso le modifiche non siano possibili
- Modificare lato database i valori dell'utente con quelli nuovi
- Creare un pulsante “manage” che apre un modulo per la modifica del nome utente e dello username
- Mostrare a video un messaggio di errore nel caso i nuovi valori siano già in uso e devono essere cambiati
- Creare un pulsante “save” per inviare i nuovi dati al backend
- Implementazione di messaggi di errore nel caso non sia avvenuto con successo la modifica
- Il sistema deve aggiornare in modo dinamico le modifiche
- Verificare che effettivamente le modifiche siano avvenute con successo
- Verificare che gli errori siano visualizzati correttamente, e che non possano esserci duplicati nel database a causa delle modifiche

Difficoltà



Tempo

1 settimana

User story 12: elimina attività segnalate (dashboard admin)

Come admin, voglio poter visualizzare le attività segnalate, ordinate in modo decrescente (da quelle con maggiori segnalazioni a quelle con minori segnalazioni), e poterle eliminare nel caso non rispettino le linee guida

Criteri di accettazione

- L'admin può visualizzare un elenco di attività segnalate, ordinate in modo decrescente in base al numero di segnalazioni ricevute
- L'admin può controllare i dati dell'attività e decidere di eliminarla nel caso non rispettasse le linee guida del sito
- Se l'attività viene eliminata con successo, questa viene rimossa dal sistema e non è più visibile né ricercabile dagli utenti e dagli admin
- In caso di errore il sistema mostra un messaggio che spiega la motivazione

Tasks

- Creare un endpoint API per recuperare tutte le attività segnalate già in ordine decrescente in base al numero di segnalazioni
- Creare un endpoint API per poter eliminare dal sistema le attività segnalate che non rispettano le linee guida
- Implementare il controllo dell'autorizzazione, in modo che solamente gli utenti admin possono vedere e eliminare tutte le attività segnalate
- Aggiornare l'aggiornamento in tempo reale di tutte le attività segnalate una volta che avviene un'eliminazione
- Aggiungere una sezione nella pagina principale della dashboard admin che fa vedere tutte le attività in ordine decrescente
- Implementare la visualizzazione di tutti i dati di ogni attività segnalata, in primi il numero di segnalazioni per poter comprendere quanti utenti l'hanno segnalata
- Creare un bottone accanto ad ogni attività segnalata denominato "delete" in modo che un admin possa eliminare l'attività
- mostrare messaggi di errore o di successo per l'eliminazione di una attività
- Testare che le attività siano visualizzate in modo decrescente per numero di segnalazioni
- Controllare che si vedano solamente le attività con segnalazioni maggiori di 0
- Testare che solamente gli admin possano eliminare le attività segnalate
- Testare il pulsante elimina e controllare che l'aggiornamento delle attività avvenga correttamente
- Testare gli errori e i messaggi di errore

Difficoltà



Tempo

4-5 giorni

User story 13: Promuovi user ad admin (dashboard admin)

Come admin voglio avere una lista di tutti gli user non admin, e la possibilità per mezzo di una searchbar di filtrarli per username, in modo poi di poterli promuovere ad admin e dare loro permessi più elevati

Criteri di accettazione

- L'admin può visualizzare una lista di tutti gli utenti non admin presenti nel sistema
- L'admin può filtrare per username gli utenti, per mezzo di una searchbar
- Una volta trovato l'utente che ci interessa, se gli premiamo accanto il tasto promuovi possiamo cambiare il ruolo all'utente, da user ad admin

- La lista viene aggiornata in tempo reale dopo la promozione, e l'utente promosso non è né visibile né ricercabile
- Se si presenta un errore durante il processo di promozione, il sistema fornisce un messaggio di errore che spiega il motivo dell'inconveniente

Tasks

- Creare un endpoint API che restituisca tutti gli utenti non admin
- Creare un endpoint API che mi permetta di cambiare ruolo da user a admin
- Creare un endpoint API che mi restituisca gli utenti filtrati per username
- Assicurarsi che il sistema aggiorni correttamente i permessi dell'utente nel database, promuovendolo ad admin
- Creare una sezione nella dashboard admin che mi permette di visualizzare tutti gli utenti non admin
- Creare una searchbar nella dashboard admin che mi permette di filtrare tutti gli user in base allo username
- Aggiornare la lista degli utenti in tempo reale
- Implementare i messaggi di errore nel caso la promozione non sia avvenuta con successo
- Verificare che l'admin possa visualizzare tutti gli user
- Verificare che l'admin possa promuovere gli user
- Verificare che il filtraggio avvenga in modo corretto
- Testare che l'aggiornamento degli user avvenga in tempo reale, e che io non possa cercare user che erano utenti semplici promossi a admin
- Simulare un errore e osservare i messaggi di errore

Difficoltà



Tempo

1 settimana

User story 14: Elimina user sospetto (dashboard admin)

Come admin voglio avere un bottone elimina utente accanto alle attività segnalate, in quanto se ritengo sospetta le attività di quello user sull'applicazione posso decidere di eliminargli l'account

Criteri di accettazione

- L'admin può eliminare l'account di un utente per sospetto utilizzo improprio dell'applicazione per mezzo di un pulsante
- Quando l'admin elimina un utente, tutte le attività create da quell'utente vengono rimosse dal sistema

- Il sistema aggiorna in tempo reale le attività, non mostrando e non potendo cercare le attività cancellate a cascata dopo l'eliminazione dell'utente

Tasks

- Creare un endpoint API che permetta all'admin di eliminare ogni user, e eliminare a cascata dal database tutte le attività che hanno come creatore lo user eliminato
- Assicurarsi che il sistema aggiorni correttamente le attività e gli users nel database
- Creare un bottone nella dashboard admin, accanto alle attività segnalate, che permettono agli admin di eliminare l'account di un utente
- Aggiornare la lista degli utenti in tempo reale
- Aggiornare le attività in tempo reale
- Implementare i messaggi di errore nel caso la cancellazione non sia avvenuta con successo, e l'eliminazione di tutte le attività correlate non sia avvenuta con successo
- Verificare che l'admin possa eliminare uno user
- Verificare che l'aggiornamento di attività e users avvenga in modo corretto
- Testare che l'aggiornamento degli user avvenga in tempo reale, e che non sia più presente l'admin e le sue attività
- Simulare un errore e osservare i messaggi di errore

Difficoltà



Tempo

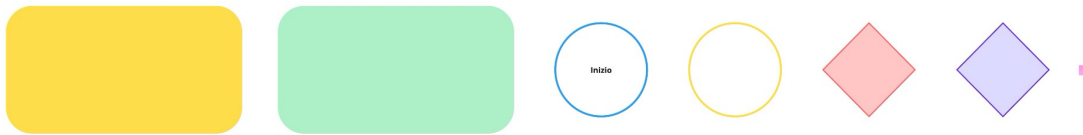
1 settimana

2. User Flows

In questa sezione del documento di sviluppo vengono riportati gli user flows per il ruolo di utente e admin della nostra applicazione AroundYou.

Qua sotto è presente una piccola legenda per poter leggere la corretta lettura del diagramma.

- rettangolo arancione : risposte del sistema
- rettangolo verde : azioni dell'utente
- cerchio blu : inizio
- cerchio giallo : pausa
- rombo rosso : decisioni dell'utente
- rombo viola : decisioni del sistema
- quadratini piccoli : associazioni di percorsi



Un utente generico (autenticato e non autenticato) può fare una ricerca utilizzando la barra di ricerca oppure i filtri prestabili. Una volta che le attività sono state filtrate, e sono stati ottenuti risultati, l'utente ha la possibilità di scorrere le attività dalla lista presente sulla mappa, e da quest'ultima può osservare i vari ping che rappresentano la posizione specifica delle attività di ogni evento. Al click su una attività della lista si apre un banner che presenta tutte le informazioni utili elargite dal creatore. Il banner oltretutto presenta anche dei bottoni: salva e partecipa che possono essere premuti solamente da utenti autenticati. Se viene premuto partecipa (da utente autenticato), questa attività viene salvata, il numero di slot disponibili diminuisce e la partecipazione è visibile al creatore, oltre che l'attività viene salvata nella zona personale dell'utente "salvati".

Se viene premuto il tasto segnala invece, l'attività verrà segnalata agli admin, che potranno visionarla dalla loro pagina personale.

Un utente quindi deve avere un profilo per sbloccare funzionalità aggiuntive, come la creazione di una attività. In ogni momento, un utente non registrato può creare un profilo, premendo sull'icona del profilo in alto a destra (presente in ogni pagina). Viene indirizzato alla pagina di registrazione, dove gli si presenterà un form da compilare o può ulteriormente premere il bottone di Google per registrarsi con un profilo Google già esistente. In entrambi i casi, solamente se le informazioni inserite sono valide il sistema creerà un nuovo profilo, in caso contrario verrà segnalato all'utente l'errore, il quale potrà riprovare la registrazione. Se un utente ha già un profilo, dalla stessa icona può entrare nell'area di log-in. Anche in questo caso la pagina presenta un form da compilare oppure un bottone di Google per poter autenticarsi con un profilo già registrato di Google. Nel caso di dati errati, il sito permette all'utente di provare nuovamente ad autenticarsi.

Una volta che l'utente è autenticato può creare una nuova attività o entrare nella propria area personale.

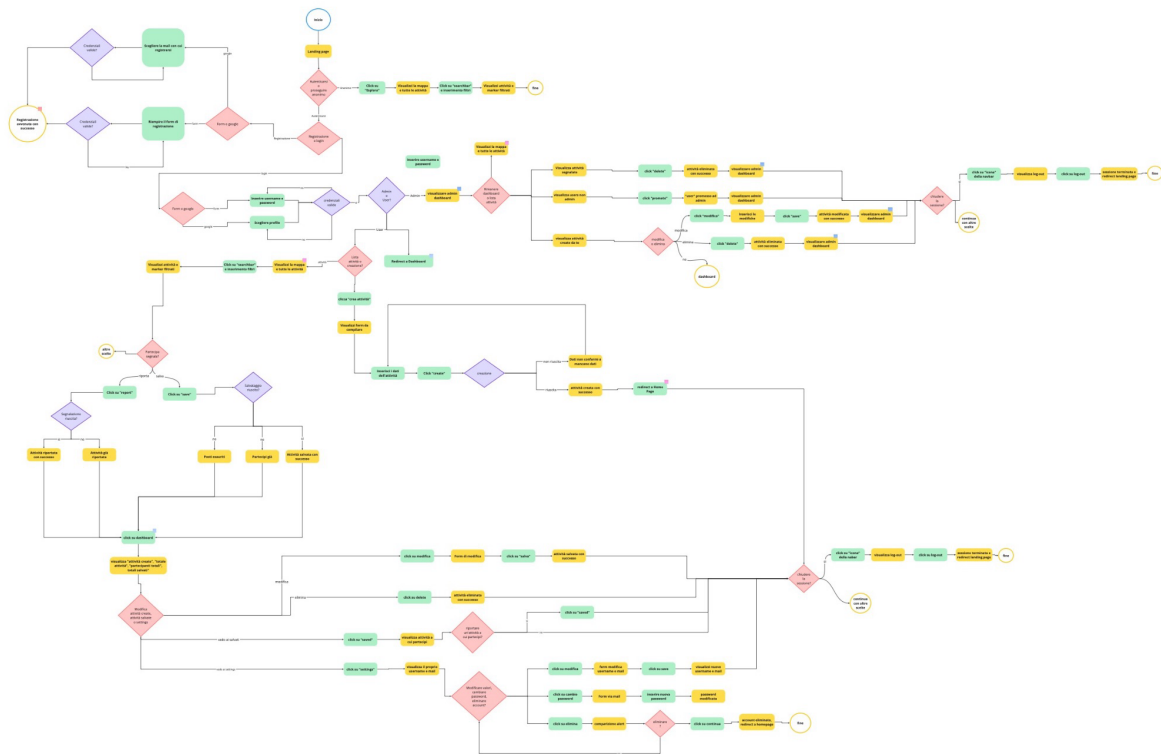
Per creare una nuova attività deve premere il bottone sulla navbar che lo porta alla pagina di creazione, nella quale è presente un form nel quale bisogna inserire tutte le informazioni richieste: nome, luogo, topics, data, partecipanti massimi.

Se non sono inseriti tutti i dati, il form non può essere inviato.

Una volta creata l'attività questa è visibile nella pagina principale del sito e nella pagina personale del creatore.

Per entrare nella pagina personale, una volta eseguita l'autenticazione, bisogna premere sull'icona, che in precedenza serviva ad entrare nelle pagine di login o registrazione, in quanto il banner si modifica dopo l'autenticazione. Troviamo ora i bottoni dashboard e logout (nel caso l'utente volesse scollegare il profilo). La prima pagina della zona personale per un utente semplice presenta un banner con il numero di attività create totali dalla creazione del profilo e il numero di partecipazioni totali e una lista di attività create, che se non sono terminate, possono essere modificate o eliminate. Per mezzo di alcuni bottoni fissi, l'utente può navigare nell'area personale. La seconda zona dell'area personale è quella riguardante la lista di tutte le attività segnalate. E infine troviamo le impostazioni, dove l'utente può modificare i dati di accesso ed eliminare il proprio profilo. L'area personale dell'admin è differente: è una unica pagina con vari blocchi modulari che presentano le attività segnalate ordinate per numero di segnalazioni in modo da poter esser eliminate, la lista di utenti i quali possono essere promossi admin, e le proprie attività create.

In ogni momento gli utenti possono premere sul logo del sito per poter tornare alla pagina principale.



Per poter vedere il flow in modo dettagliato si può seguire il link proposto, il quale porta alla pagina del sito web Miro: [User Flow](#)

3. Web Apis

L'API è stata progettata seguendo le specifiche di OpenAPI 3.0 e il paradigma RESTFUL.

L'API utilizza l'autenticazione tramite token, e negli endpoint in cui serve autenticazione l'header "autheticateToken" indica il token valido da passare.

La specifica delle API è invece disponibile nel repository GitHub al link:

Le riportiamo in seguito:

```
openapi: 3.0.0

info:
  version: '2.0'
  title: "AroundYou OpenAPi 3.0.0"
```

```

description: API for managing AroundYou requests

license:

  name: MIT

servers:

- url: http://localhost:8000/api/v2

  description: Backend

paths:

  /activities:

    get:

      description: Get the activities from the system.

      summary: Get all the activities.

      responses:

        '200':

          description: 'Collection of activities'

          content:

            application/json:

              schema:

                type: array

                items:

                  $ref: '#/components/schemas/Activity'

  /activities/{query}:

    get:

      parameters:

        - name: query

          in: path

```



```

      required: true

      schema:

        type: string

      description: Keyword to search activities

description: Search a list of activities with a keyword

summary: Search activities

responses:

  '200':

    description: 'Collection of activities'

    content:

      application/json:

        schema:

          type: array

          items:

            $ref: '#/components/schemas/Activity'

/join/{id}:

  put:

    parameters:

      - name: authenticateToken

        in: header

        schema:

          type: string

      - name: id

        in: path

        required: true

```

```

    schema:
      type: string
      description: 'Id of the activity to join'
  description: 'Join an activity'
  summary: 'Join activity'
  responses:
    '200':
      description: 'Activity joined successfully'
    '400':
      description: 'ActivityId is not valid'
    '401':
      description: 'Invalid token or credentials'
    '404':
      description: 'Activity ended or already joined'
    '500':
      description: 'Error on activity join'
/join:
  get:
    parameters:
      - name: authenticateToken
        in: header
        schema:
          type: string
    description: 'Get the list of the joined activities'
    summary: 'Get joined activities'

```

```
responses:

  '200':

    description: 'Collection of joined activities'

    content:

      application/json:

        schema:

          type: array

          items:

            $ref: '#/components/schemas/Activity'

  '401':

    description: 'Invalid token or credentials'

  '500':

    description: 'Server error'

/report/{id}:

  put:

    parameters:

      - name: authenticateToken

        in: header

        schema:

          type: string

      - name: id

        in: path

        required: true

        schema:

          type: string
```

```

    description: 'Id of the activity to report'

description: 'Report an activity'

summary: 'Report activity'

responses:

  '200':

    description: 'Activity reported succesfully'

  '400':

    description: 'Activity already reported'

  '401':

    description: 'Invalid token or credentials'

  '404':

    description: 'Activity not found'

  '500':

    description: 'Error on activity report'

/admin/manageActivities:

get:

  parameters:

    - name: authenticateToken

      in: header

      schema:

        type: string

    description: 'Get a list of activities with warnings sorted by the most
warned'

    summary: 'Get activities with warnings'

    responses:

```

```

'200':
  description: 'Collection of activities'
  content:
    application/json:
      schema:
        type: array
        items:
          $ref: '#/components/schemas/Activity'
'204':
  description: 'No activity with warnings found'
'401':
  description: 'Invalid token or credentials'
'403':
  description: 'User is not an admin'
'500':
  description: 'Server error'
/admin/manageActivities/{id}:
  delete:
    parameters:
      - name: authenticateToken
        in: header
        schema:
          type: string
      - name: id
        in: path

```

```

      required: true

      schema:

        type: string

        description: 'Id of the activity to remove'
    description: 'Remove an activity'
    summary: 'Remove activity'
    responses:

      '200':

        description: 'Activity removed succesfully'

      '401':

        description: 'Invalid token or credentials'

      '403':

        description: 'User is not an admin'

      '404':

        description: 'Activity not found'

      '500':

        description: 'Error during removal'
/admin/manageUsers/{query}:

get:

  parameters:

    - name: authenticateToken

      in: header

      schema:

        type: string

    - name: query

```

```

    in: path

    required: true

    schema:

      type: string

    description: 'Username to search for users'

  description: 'Search users by username'

  summary: 'Search users by username'

  responses:

    '200':

      description: 'Collection of users'

      content:

        application/json:

          schema:

            type: array

            items:

              $ref: '#/components/schemas/User'

    '401':

      description: 'Invalid token or credentials'

    '403':

      description: 'User is not an admin'

    '500':

      description: 'Server error'

/admin/manageUsers/{id}:

  put:

    parameters:

```

```
- name: authenticateToken

  in: header

  schema:

    type: string

- name: id

  in: path

  required: true

  schema:

    type: string

  description: 'Id of the user to promote to admin'

description: 'Promote a user to admin'

summary: 'Promote user'

responses:

  '200':

    description: 'User promoted succesfully'

  '400':

    description: 'User is already an admin'

  '401':

    description: 'Invalid token or credentials'

  '403':

    description: 'User is not an admin'

  '404':

    description: 'User not found'

  '500':

    description: 'Error during promotion'
```



```
delete:

  parameters:

    - name: authenticateToken

      in: header

      schema:

        type: string

  description: 'Remove a user'

  summary: 'Remove user'

  responses:

    '200':

      description: 'User removed succesfully'

    '400':

      description: 'User is an admin'

    '401':

      description: 'Invalid token or credentials'

    '403':

      description: 'User is not an admin'

    '404':

      description: 'User not found'

    '500':

      description: 'Error during removal'

/admin/manageUsers:

  get:

    parameters:

      - name: authenticateToken
```

```

    in: header

    schema:

      type: string

description: 'Return the list of users'

summary: 'Get users'

responses:

  '200':

    description: 'Collection of users'

    content:

      application/json:

        schema:

          type: array

          items:

            $ref: '#/components/schemas/User'

  '401':

    description: 'Invalid token or credentials'

  '403':

    description: 'User is not an admin'

  '500':

    description: 'Server error'

/users/activities:

get:

  parameters:

    - name: authenticateToken

    in: header

```

```

    schema:

      type: string

    description: 'Get the list of activities created by the user'

    summary: 'Get activities'

    responses:

      '200':

        description: 'Collection of activities'

        content:

          application/json:

            schema:

              type: array

              items:

                $ref: '#/components/schemas/Activity'

      '204':

        description: 'No activities found'

      '400':

        description: 'Activity fields are missing'

      '401':

        description: 'Invalid token or credentials'

      '500':

        description: 'Server error'

  post:

    parameters:

      - name: authenticateToken

        in: header

```

```
    schema:

      type: string

    description: 'Post an activity'

    summary: 'Post activity'

    requestBody:

      required: true

      content:

        application/json:

          schema:

            $ref: "#/components/schemas/Activity"

  responses:

    '201':

      description: 'Activity posted'

      content:

        application/json:

          schema:

            type: object

            items:

              $ref: '#/components/schemas/Activity'

    '400':

      description: 'Activity fields are missing'

    '401':

      description: 'Invalid token or credentials'

    '500':

      description: 'Error on creation'
```

```
/users/activities/{id}:

put:

  parameters:

    - name: authenticateToken

      in: header

      schema:

        type: string

    - name: id

      in: path

      required: true

      schema:

        type: string

      description: 'Id of the activity to update'

  description: 'Update an activity'

  summary: 'Update activity'

  requestBody:

    required: true

    content:

      application/json:

        schema:

          $ref: "#/components/schemas/Activity"

  responses:

    '200':

      description: 'Activity updated succesfully'

    '401':
```

```

        description: 'Invalid token or credentials'

    '404':
        description: 'Activity not found'

    '500':
        description: 'Error on activity update'

delete:

    parameters:

        - name: authenticateToken

          in: header

          schema:

            type: string

        description: 'Delete an activity'

        summary: 'Delete activity'

        responses:

            '200':

                description: 'Activity deleted succesfully'

            '401':

                description: 'Invalid token or credentials'

            '404':

                description: 'Activity not found or invalid ActivityId'

            '500':

                description: 'Error during activity delete'

/users/private:

    get:

        parameters:

```

```
- name: authenticateToken

  in: header

  schema:

    type: string

description: 'Get the user informations'

summary: 'Get user informations'

responses:

  '200':

    description: 'User informations'

    content:

      application/json:

        schema:

          $ref: '#/components/schemas/User'

  '401':

    description: 'Invalid token or credentials'

  '404':

    description: 'User not found'

  '500':

    description: 'Server error'

put:

  parameters:

    - name: authenticateToken

      in: header

      schema:

        type: string
```

```
description: 'Update the user informations'

summary: 'Update user informations'

requestBody:

  required: true

  content:

    application/json:

      schema:

        $ref: "#/components/schemas/User"

responses:

  '200':

    description: 'User informations updated'

  '400':

    description: 'Field not found'

  '401':

    description: 'Invalid token or credentials'

  '404':

    description: 'User not found'

  '409':

    description: 'Username or mail already in use'

  '500':

    description: 'Server error'

delete:

  parameters:

    - name: authenticateToken

      in: header
```



```
    schema:

      type: string

    description: 'Delete the user and his activities'

    summary: 'Delete user'

    responses:

      '200':

        description: 'User deleted succesfully'

      '401':

        description: 'Invalid token or credentials'

      '500':

        description: 'Server error'

/auth/login:

  post:

    description: 'Login to the system'

    summary: 'Login'

    requestBody:

      required: true

      content:

        application/json:

          schema:

            type: object

            required:

              - username

              - password

            properties:
```

```
    username:

      type: string

      description: 'Username of the user'

    password:

      type: string

      description: 'Password of the user'

  responses:

    '200':

      description: 'Login successful'

      content:

        application/json:

          schema:

            type: object

            properties:

              token:

                type: string

                description: 'Token for authentication'

    '401':

      description: 'Invalid credentials'

    '500':

      description: 'Error on login'

/auth/register:

  post:

    description: 'Register to the system'

    summary: 'Register'
```

```
requestBody:

  required: true

  content:

    application/json:

      schema:

        $ref: "#/components/schemas/User"

responses:

  '201':

    description: 'User registered'

  '400':

    description: 'Missing credentials'

  '409':

    description: 'Username or email already in use'

  '500':

    description: 'Error during the creation of the user'

components:

  schemas:

    Activity:

      type: object

      required:

        - name

        - topic

        - place

        - date
```

```
- creator

- maxSlot

properties:

  name:

    type: string

    description: 'Name of the activity'

  topic:

    type: array

    default: ["string"]

    description: 'Topics of the activity'

  place:

    type: string

    description: 'Place of the activity'

  date:

    type: string

    description: 'Date of the activity'

  creator:

    type: string

    description: 'Username of the creator of the activity'

  maxSlot:

    type: integer

    description: 'Number of total free slots of the activity'

  remainingSlots:

    type: integer

    description: 'Number of remaining free slots of the activity'
```

```
warnings:

  type: integer

  description: 'Number of warnings of the activity'

ended:

  type: boolean

  default: false

  description: "True if the activity is ended"

User:

  type: object

  required:

    - name

    - surname

    - email

    - username

    - password

  properties:

    name:

      type: string

      description: "Name of the user"

    surname:

      type: string

      description: "Surname of the user"

    email:

      type: string

      description: "Mail of the user"
```

```
username:

  type: string

  description: "Username of the user"

password:

  type: string

  description: "Hashed password of the user"

Admin:

  type: object

  required:

    - adminId

    - userId

  properties:

    adminId:

      type: string

      description: 'Id associated to the collection of admins'

    userId:

      type: string

      description: 'Id of the user promoted to admin'

Report:

  type: object

  required:

    - reportId

    - activityId

    - userId

  properties:
```

```

reportId:

  type: string

activityId:

  type: string

userId:

  type: string

Participation:

  type: object

required:

  - participationId

  - activityId

  - userId

properties:

  participationId:

    type: string

  activityId:

    type: string

  userId:

    type: string

```

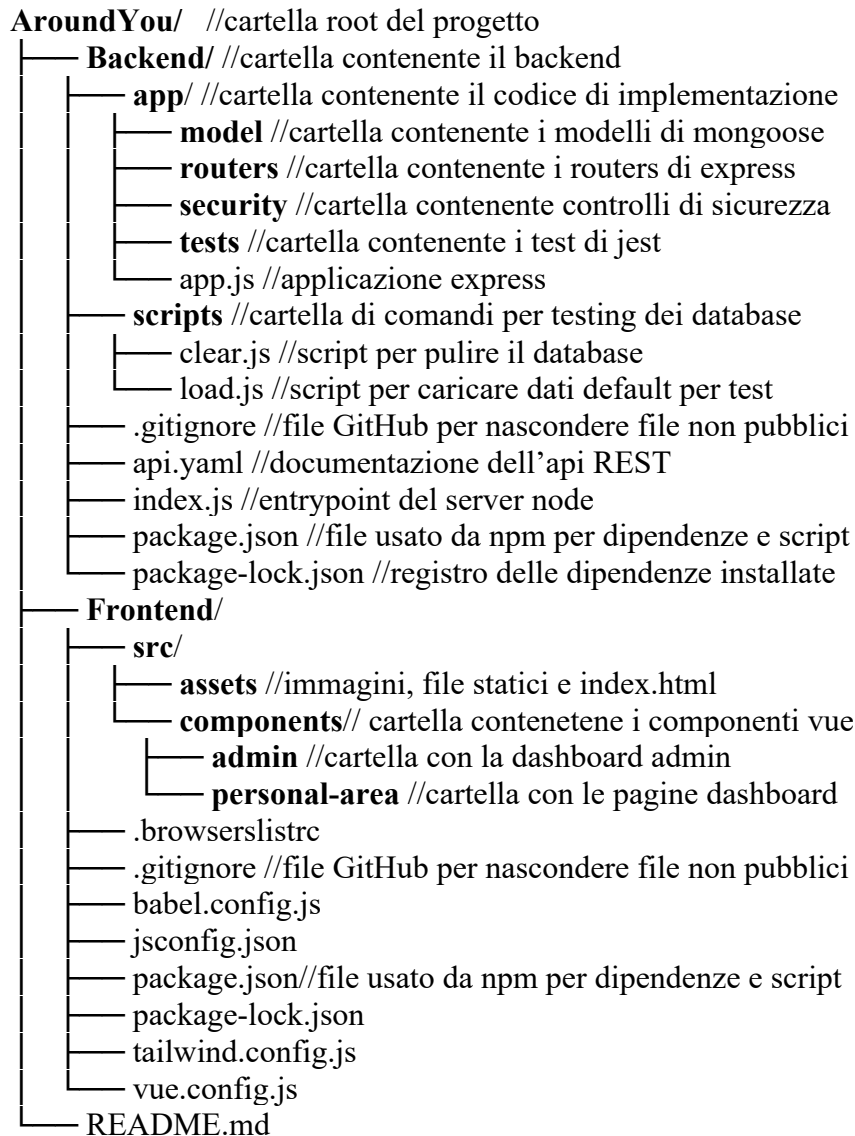
4. Implementazione

L'applicazione AroundYou è stata sviluppata utilizzando il framework NodeJS per il Backend e Vue.js per il frontend. La documentazione delle API è stata realizzata con Swagger perché offre uno standard riconosciuto e diffuso a livello internazionale per descrivere API RESTful in modo chiaro e strutturato.

La scelta di questo stack tecnologico è stata determinata dalle competenze e materiali forniti dal corso.

4.1 Repository Organization

Si riporta una dettagliata descrizione e rappresentazione del codice nelle varie directory.



4.2 Branching strategy e organizzazione del lavoro

Lo sviluppo collaborativo della web app è stato gestito attraverso una strategia di branching strutturata su GitHub, al fine di garantire ordine, tracciabilità delle modifiche e facilitare l'integrazione del lavoro di gruppo.

Abbiamo adottato una struttura gerarchica dei rami così organizzata:

- **main**: rappresenta il ramo principale e stabile, in cui è presente la versione completata e consolidata dell'applicazione. Le modifiche vengono integrate in questo ramo solo quando sono state completamente sviluppate, testate e verificate.
- Rami di sviluppo individuali: ciascun membro del team ha lavorato su un ramo dedicato, rispettivamente **development_dani**, **development_fra** e **development_ale**. Su questi rami ciascuno ha potuto sviluppare in autonomia i componenti assegnati, effettuando regolarmente commit e push delle modifiche locali.
- Ramo **development** comune: per coordinare l'integrazione delle funzionalità sviluppate individualmente, è stato introdotto un ramo intermedio chiamato development. Periodicamente, ciascun membro effettuava un merge del proprio ramo nel **development**, dove venivano risolti eventuali conflitti. Questo passaggio assicurava che il codice fosse compatibile e funzionante prima di essere reintegrato nei rami individuali, così da consentire a tutti di ripartire da una base aggiornata e condivisa.
- Ramo di **refactor**: nelle fasi finali del progetto è stato creato un ramo specifico denominato refactor, dedicato al miglioramento della qualità del codice e alla riorganizzazione del database. In questo ramo, senza aggiungere nuove funzionalità, il team si è concentrato sulla ristrutturazione del codice (rimozione di duplicazioni, semplificazione della logica, ridefinizione della struttura dei file e dei moduli). È stata anche eseguita una riorganizzazione della struttura del database (modifica di tabelle, ottimizzazione dei rapporti tra entità, migrazione di dati, normalizzazione e miglioramento delle performance). Oltre a ciò abbiamo anche fatto un potenziamento della leggibilità e della manutenibilità generale del progetto.

Questa strategia di branching ci ha permesso di lavorare in parallelo evitando conflitti e sovrapposizioni. Ogni membro ha potuto concentrarsi sul proprio ramo individuale mantenendo l'autonomia necessaria per lo sviluppo, ma con la consapevolezza di dover periodicamente integrare il proprio lavoro nel ramo development condiviso, in cui veniva effettuata una verifica incrociata del codice da parte degli altri membri tramite pull request e code review. Questo processo ha favorito un controllo di qualità continuo e una diffusione omogenea delle conoscenze tecniche all'interno del gruppo.

Per garantire un'organizzazione efficace e professionale del lavoro, abbiamo adottato una strategia collaborativa basata su una pianificazione precisa e una suddivisione chiara dei compiti, in linea con le best practice dello sviluppo software agile. All'inizio di ogni sprint, il team si è riunito per definire gli obiettivi da raggiungere e assegnare in modo equo le responsabilità a ciascun membro. Ogni sviluppatore ha avuto l'incarico di realizzare

specifiche funzionalità o moduli, concordati durante la fase di pianificazione. La comunicazione interna è stata costante ed efficiente, soprattutto grazie alla possibilità di lavorare assieme fisicamente.

4.3 Dependencies

Il progetto è basato sull'ambiente Node.js e sul package manager npm.

Abbiamo utilizzato librerie standard con estensiva documentazione per garantire ulteriore sicurezza al progetto e permetterci una migliore implementazione.

Abbiamo utilizzato nel backend:

- **bcrypt** e **bcryptjs** per garantire sicurezza delle password tramite hashing e salting
- **cors** per implementare il meccanismo di Cross-origin resource sharing
- **dotenv** per l'utilizzo di file d'ambiente privato contenente le credenziali del database
- **express** per la gestione delle richieste tramite server dedicato
- **jsonwebtoken** per garantire l'autenticazione attraverso token
- **mongoose** per interfacciarsi al database mongodb
- **jest** e **supertest** per testare le richieste e risposte di express

Mentre nel frontend:

- **vuepick/vue-datepicker** per l'inserimento di data e orario delle attività
- **axios** per fare richieste all'API
- **mitt** per condividere dati tra componenti del frontend
- **leaflet** per l'implementazione della mappa e dei ping
- **vuejs** per la creazione del server frontend
- **tailwindcss** come framework per la parte grafica

4.4 Database

Di seguito viene descritta l'architettura dati adottata con MongoDB, illustrando le motivazioni principali, il disegno degli schemi (collections), i vincoli di integrità, gli indici e le relazioni fra documenti.

- **Collection Activity**

Raccoglie tutte le attività / eventi disponibili nell'applicazione.

```
"name": { type: String, required: true },
"topic": { type: [String], required: true },
"place": { type: String, required: true },
```

```
"date": { type: Date, required: true },
"creator": { type: String, required: true },
"maxSlot": { type: Number, required: true },
"remainingSlots": Number,
"warnings": { type: Number, default: 0 },
"ended": { type: Boolean, default: false }
```

Name: il titolo dell'attività.

Topic: array contenente tutti i topic relativi all'attività.

Place: L'indirizzo geografico in cui si svolgerà l'attività.

Date: La data in cui si svolgerà l'attività.

Creator: Lo username del creatore dell'attività.

maxSlot: Il massimo numero di posti disponibili per l'attività.

remainingSlots: Il numero di posti liberi rimanenti per iscriversi all'attività.

warnings: Il contatore delle segnalazioni che quell'attività ha ricevuto.

ended: flag che segnala se l'evento è già concluso.

- **Collection User**

Contiene il profilo di ciascun utente registrato nel sistema.

```
name: { type: String, required: true },
surname: { type: String, required: true },
email: { type: String, required: true, unique: true },
username: { type: String, required: true, unique: true },
password: { type: String, required: true },
```

Name: Il nome dell'utente.

Surname: Il cognome dell'utente.

Email: L'indirizzo di posta dell'utente, deve essere unico all'interno del sistema.

Username: Appellativo scelto dall'utente in fase di registrazione, anch'esso unico.

Password: La password dell'utente, necessaria per il login.

- **Collection Admin:**

Definisce quali utenti hanno privilegi amministrativi all'interno dell'applicazione. Piuttosto di inserire un campo "isAdmin" direttamente nello user document, abbiamo isolato i privilegi in una collection dedicata. Ciò favorisce una separazione delle responsabilità (principio di single responsibility) e facilita eventuali estensioni: in futuro potremmo associare a ciascun admin permessi differenziati (es : "moderatore", "super-admin") aggiungendo altri campi.

```
const adminSchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, required:
true, unique: true }
});
```

Il riferimento "userId" serve a indicare che all'admin sono collegati anche tutti i dati tipici di uno user normale (username, email...). Inoltre il riferimento è unico perché così si evita che uno user sia più volte registrato nel sistema.

- **Collection Report**

Registra le registrazioni effettuate dagli utenti su particolari attività (ad esempio per contenuti inappropriati o problemi organizzativi). Essendo una relazione many-to-many (molti utenti possono segnalare la stessa attività, e un utente può fare più segnalazioni), si è optato per una collection dedicata.

```
  userId: { type: mongoose.Schema.Types.ObjectId, required: true
},
  activityId: { type: mongoose.Schema.Types.ObjectId, required:
true },
  reportedAt: { type: Date, default: Date.now }
```

userId: l'ID dell'utente che ha effettuato la segnalazione.

activityId: l'ID dell'attività segnalata.

reportedAt: Momento in cui la segnalazione è avvenuta. Tramite Date.now assicura che il timestamp venga fissato automaticamente al momento della registrazione.

- **Collection Participation**

Tiene traccia delle iscrizioni degli utenti alle attività: chi si è iscritto a cosa e quando.

Anche in questo caso si tratta di una relazione many-to-many, perciò serve uno schema dedicato.

```

    userId: { type: mongoose.Schema.Types.ObjectId, required: true
  },

    activityId: { type: mongoose.Schema.Types.ObjectId, required:
true },

    date: { type: Date, default: Date.now }

```

userId: l'ID dell'utente che si è iscritto all'attività.

activityId: l'ID dell'attività a cui un userId ha partecipato.

date: Il momento in cui userId ha effettuato la partecipazione ad activityID.

5. Testing

La testsuite di tutte le funzioni di ogni router di express.js corrispondenti ad altrettanti endpoint dell'API è stata scritta sui relativi file di test con estensione .test.js, sotto un'unica cartella /tests.

L'esecuzione e il controllo di ogni caso è possibile attraverso le librerie di jest e supertest che offrono, oltre al rilevamento dei test superati anche un tool per visualizzare quali linee di codice sono state ispezionate durante il testing per facilitare l'implementazione e l'eventuale mancanza di ulteriori casi di test.

Numero Test Case	Descrizione Test Case	Test Data	Precondizioni	Risultato Atteso	Risultato riscontrato	Note:
GET api/v2/activities						

1	Restituizi one di tutte le attività	Nessuno	Almeno un'attività già presente nel sistema	Viene restituito un array di tutte le attività presenti nel sistema e lo stato 200 OK	Risultato atteso. 200 OK	
2	Richiesta di visualizz are attività senza attività presenti nel sistema	Nessuno	Nessuna attività presente nel sistema	Viene restituito un array di attività vuoto e un messaggio "Nessuna attività trovata" e lo stato 200 OK	Risultato atteso. 200 OK	
GET api/v2/ac tivities/{ query}						
1	Ricerca attività con una stringa di ricerca qualsiasi	Nessuno	Almeno un'attività già presente nel sistema	Viene restituito un array di tutte le attività presenti nel sistema filtrate per la stringa di ricerca e lo stato 200 OK	Risultato atteso. 200 OK	
2	Ricerca attività con un id attività	id	Almeno un'attività già presente nel sistema	Viene restituito un array di tutte le attività presenti nel sistema filtrate per id e lo stato 200 OK	Risultato atteso. 200 OK	

3	Ricerca attività con un nome attività	nome	Almeno un'attività già presente nel sistema	Viene restituito un array di tutte le attività presenti nel sistema filtrate per nome attività e lo stato 200 OK	Risultato atteso. 200 OK	
4	Ricerca attività con un topic	topic	Almeno un'attività già presente nel sistema	Viene restituito un array di tutte le attività presenti nel sistema filtrate per topic e lo stato 200 OK	Risultato atteso. 200 OK	
5	Ricerca attività con una posizione	posizione	Almeno un'attività già presente nel sistema	Viene restituito un array di tutte le attività presenti nel sistema filtrate per posizione e lo stato 200 OK	Risultato atteso. 200 OK	
6	Ricerca attività con un creatore	topic	Almeno un'attività già presente nel sistema	Viene restituito un array di tutte le attività presenti nel sistema filtrate per creatore e lo stato 200 OK	Risultato atteso. 200 OK	
7	Ricerca attività con una query qualsiasi	Nessuno	Nessuna attività presente nel sistema	Viene restituito un array di attività vuoto e un messaggio "Nessuna attività trovata" e lo stato 200 OK	Risultato atteso. 200 OK	
POST api/v2/au						

th/register						
1	Registrarsi al servizio con i dati richiesti dal sistema	nome, cognome, email, username, password	Utente non registrato al servizio con	L'utente viene registrato nella tabella utenti, viene restituito il messaggio "Utente registrato con successo" e viene inviato lo stato 201 Created	Risultato atteso. 201 Created	
2	Registrarsi al servizio senza tutti i dati richiesti dal sistema	nome, cognome, email, username, password	Esiste un utente già registrato al servizio con la stessa mail o username	L'utente non viene registrato nella tabella utenti e viene restituito il messaggio "Nome utente o email già in uso" con lo stato 409 Conflict	Risultato atteso. 409 Conflict	
POST api/v2/auth/login						
1	Accedere al servizio con i dati richiesti dal sistema	username, password	Esiste un utente già registrato al servizio con le credenziali corrispondenti	L'utente ottiene l'accesso al suo account e viene restituito un accessToken e il codice di stato 200 OK	Risultato atteso 200 OK	

2	Accedere al servizio senza i dati richiesti dal sistema	username, password	Esiste un utente già registrato al servizio senza le credenziali corrispondenti	L'utente non ottiene l'accesso all'account e viene restituito il codice di stato 401 Unauthorized	Risultato atteso 401 Unauthorized	
3	Accedere al servizio senza i dati richiesti dal sistema	username, password	Non esiste un utente già registrato al servizio con le credenziali corrispondenti	L'utente non ottiene l'accesso all'account e viene restituito il codice di stato 401 Unauthorized	Risultato atteso 401 Unauthorized	
PUT api/v2/join/{id}						
1	Partecipare ad un'attività dato l'idAttività corrispondente	idAttività, accessToken,	L'utente ha eseguito l'accesso e possiede un accessToken e l'idAttività inserito è corretto e corrisponde ad un'attività esistente non terminata	La partecipazione dell'utente all'attività viene registrata con successo e viene ritornato il codice di stato 200 OK	Risultato atteso 200 OK	
2	Partecipare ad	idAttività,	L'utente ha eseguito	La partecipazione dell'utente	Risultato atteso	

	un'attività dato l'idAttività corrispondente	accessToken,	l'accesso e possiede un accessToken e l'idAttività inserito è corretto e corrisponde ad un'attività esistente terminata	all'attività non viene registrata, lo status ended dell'attività viene settato a true e viene ritornato il codice di stato 404 Not found	404 Not found	
3	Partecipare ad un'attività dato l'idAttività corrispondente non valido	idAttività, accessToken,	L'utente ha eseguito l'accesso e possiede un accessToken e l'idAttività inserito non è corretto	La partecipazione dell'utente all'attività non viene registrata, lo status ended dell'attività viene settato a true e viene ritornato il codice di stato 400 Bad Request	Risultato atteso 400 Bad Request	
4	Partecipare ad un'attività dato l'idAttività corrispondente valido	idAttività	L'utente non ha eseguito l'accesso e non possiede un accessToken e l'idAttività inserito è corretto	La partecipazione dell'utente all'attività non viene registrata, lo status ended dell'attività viene settato a true e viene ritornato il codice di stato 401 Unauthorized	Risultato atteso 401 Unauthorized	
GET api/v2/join						

1	Ottenere tutte le attività a cui l'utente partecipa	accessToken	L'utente ha eseguito l'accesso e possiede un accessToken	Viene restituito l'array di attività a cui l'utente partecipa e lo status code 200 OK	Risultato atteso 200 OK	
2	Ottenere tutte le attività a cui l'utente partecipa	Nessuno	L'utente non ha eseguito l'accesso e non possiede un accessToken	Non viene restituito nessun dato e viene mandato lo status code 401 Unauthorized	Risultato atteso 401 Unauthorized	
POST api/v2/report/{id}						
1	Segnalar e un'attività a dato un idAttività valido	idAttività, accessToken	L'utente ha eseguito l'accesso e possiede un accessToken e l'idAttività corrisponde ad un'attività esistente e non già segnalata dall'utente	Viene incrementato il numero di segnalazioni dell'attività e viene restituito il messaggio "Segnalazione aggiunta con successo" e lo status code 200 OK	Risultato atteso 200 OK	
2	Segnalar e un'attività a dato un idAttività valido	idAttività, accessToken	L'utente ha eseguito l'accesso e possiede un accessToken e l'idAttività corrisponde ad	Non viene incrementato il contatore delle segnalazioni e viene restituito lo status	Risultato atteso 400 Bad Request	

			un'attività esistente e già segnalata dall'utente	code 400 Bad Request		
3	Segnalare un'attività dato un idAttività non valido	idAttività, accessToken	L'utente ha eseguito l'accesso e possiede un accessToken e l'idAttività non corrisponde ad un'attività esistente	Non viene incrementato il contatore delle segnalazioni e viene inviato il messaggio "Attività non trovata" e lo status code 404 Not Found	Risultato atteso 404 Not Found	
GET api/v2/users/activities						
1	Ottenere le attività create dall'utente dato un accessToken e almeno un'attività creata	accessToken	L'utente ha eseguito l'accesso e possiede un accessToken	Viene restituito un array di activities che possiedono il campo creator uguale all'username dell'utente loggato e lo status code 200 OK	Valore atteso 200 OK	
2	Ottenere le attività create dall'utente dato un accessToken	accessToken	L'utente ha eseguito l'accesso e possiede un accessToken	Viene restituito il messaggio "Ancora nessuna attività creata" e lo status	Valore atteso 204 No Content	

	ken e nessun'attività creata			code 204 No Content		
3	Ottenere le attività create dall'utente e senza un accessToken	Nessuno	L'utente non ha eseguito l'accesso e non possiede un accessToken	Viene ritornato il codice errore 401 Unauthorized	Valore atteso 401 Unauthorized	
POST api/v2/users/activities						
1	Creare un'attività dato un accessToken e tutti i campi obbligatori di un'attività	accessToken, name, topic, place, date, maxSlot, contacts	L'utente ha eseguito l'accesso e possiede un accessToken e i campi inseriti nel body sono validi	Viene creata una nuova attività con i campi uguali a quelli inviati dall'utente e viene restituita l'attività creata ed il codice di stato 201 Created	Valore atteso 201 Created	
2	Creare un'attività dato un accessToken ma non tutti i	accessToken, name, topic, place, date,	L'utente ha eseguito l'accesso e possiede un accessToken e non tutti i campi	Viene restituito il messaggio "Tutti i campi sono obbligatori" e mandato il codice di	Valore atteso 400 Bad Request	

	campi obbligatori di un'attività	maxSlot, contacts	inseriti nel body sono validi	stato 400 Bad Request		
3	Creare un'attività senza un accessToken	name, topic, place, date, maxSlot, contacts	L'utente non ha eseguito l'accesso e non possiede un accessToken	Viene ritornato il codice errore 401 Unauthorized	Valore atteso 401 Unauthorized	
PUT api/v2/activities/{id}						
1	Modificare un'attività creata dall'utente e dato un accessToken e un idAttività	idAttività, accessToken, updates	L'utente ha eseguito l'accesso e possiede un accessToken e ha inserito un idAttività di un'attività di cui è creator	Vengono aggiornati i dati dell'attività con quelli inseriti dall'utente e viene restituita l'attività aggiornata e il codice di stato 200 OK	Valore atteso 200 OK	
2	Modificare un'attività non creata dall'utente e dato un accessToken	idAttività, accessToken, updates	L'utente ha eseguito l'accesso e possiede un accessToken e ha inserito un idAttività di	Nessuna attività viene aggiornata e viene restituito all'utente il messaggio "Attività non trovata o modifica non autorizzata" con il	Valore atteso 404 Not Found	

	ken e un idAttività		un'attività di cui non è creator	codice di stato 404 Not Found		
3	Modificare un'attività senza un accessToken	idAttività, updates	L'utente non ha eseguito l'accesso e ha inserito un idAttività qualsiasi	Nessuna attività viene aggiornata e viene restituito il codice di stato 401 Unauthorized	Valore atteso 401 Unauthorized	
4	Modificare un'attività creata dall'utente e dato un accessToken e un idAttività non valido	idAttività, accessToken, updates	L'utente ha eseguito l'accesso e possiede un accessToken e ha inserito un idAttività non valido	Viene restituito il messaggio "Attività non trovata" e il codice di stato 404 Not Found	Valore atteso 404 Not Found	
DELETE api/v2/activities/{id}						
1	Eliminare un'attività creata dall'utente e dato un accessToken	idAttività, accessToken	L'utente ha eseguito l'accesso e possiede un accessToken e ha inserito un idAttività di	L'attività viene eliminata e viene inviato il messaggio "Attività eliminata con successo" insieme al codice di stato 200 OK	Valore atteso 200 OK	

	ken e un idAttività		un'attività di cui è creator			
2	Eliminare un'attività non creata dall'utente e dato un accessToken e un idAttività	idAttività, accessToken	L'utente ha eseguito l'accesso e possiede un accessToken e ha inserito un idAttività di un'attività di cui non è creator	Nessuna attività viene eliminata e viene restituito all'utente il messaggio "Attività non trovata o user non autorizzato" con il codice di stato 404 Not Found	Valore atteso 404 Not Found	
3	Eliminare un'attività senza un accessToken	idAttività	L'utente non ha eseguito l'accesso e ha inserito un idAttività qualsiasi	Nessuna attività viene eliminata e viene restituito il codice di stato 401 Unauthorized	Valore atteso 401 Unauthorized	
4	Eliminare un'attività creata dall'utente e dato un accessToken e un idAttività non valido	idAttività, accessToken	L'utente ha eseguito l'accesso e possiede un accessToken e ha inserito un idAttività non valido	Viene restituito il messaggio "Attività non trovata" e il codice di stato 404 Not Found	Valore atteso 404 Not Found	

GET api/v2/users/private						
1	Ottenere i dati dell'utente esclusa password dato un accessToken	accessToken	L'utente ha eseguito l'accesso e possiede un accessToken	Vengono restituiti tutti i dati appartenenti all'utente tranne la password e viene inviato il codice di stato 200 OK	Valore atteso 200 OK	
2	Ottenere i dati di un utente non esistente	Nessuno	L'utente da cui ottenere i dati non esiste	Viene restituito il messaggio "Utente non trovato" e viene inviato il codice di stato 404 Not Found	Valore atteso 404 Not Found	
3	Ottenere i dati dell'utente esclusa password senza un accessToken	Nessuno	L'utente non ha eseguito l'accesso e non possiede un accessToken	Viene restituito il codice di stato 401 Unauthorized	Valore atteso 401 Unauthorized	
PUT api/v2/users/private						

1	Aggiornare i campi email o username dell'utente e dato un accessToken e delle modifiche	accessToken, updates	L'utente ha eseguito l'accesso e possiede un accessToken e ha inserito username e/o mail validi e non duplicati	Vengono aggiornati i dati dell'utente con i nuovi dati inseriti e viene inviato il messaggio "Profilo aggiornato con successo" e il codice di stato 200 OK	Valore atteso 200 OK	
2	Aggiornare i campi email o username dell'utente e dato un accessToken e delle modifiche non valide	accessToken, updates	L'utente ha eseguito l'accesso e possiede un accessToken e ha inserito username e/o mail non validi ma duplicati	Non vengono aggiornati i dati dell'utente e viene inviato il messaggio "Username o email già in uso" e il codice di stato 409 Conflict	Valore atteso 409 Conflict	
3	Aggiornare i campi email o username dell'utente e dato un accessToken e	accessToken	L'utente ha eseguito l'accesso e possiede un accessToken e ha non ha inserito modifiche	Non viene aggiornato nessun dato e viene restituito il messaggio "Nessun campo da modificare" e il codice di stato 400 Bad Request	Valore atteso 400 Bad Request	

	nessuna modifica					
4	Aggiornare i campi email o username dell'utente e senza un accessToken	Nessuno	L'utente non ha eseguito l'accesso e non possiede un accessToken	Viene restituito il codice di stato 401 Unauthorized	Valore atteso 401 Unauthorized	
5	Aggiornare i campi email o username di un utente non esistente	Nessuno	L'utente da aggiornare non esiste	Viene restituito il messaggio "Utente non trovato" e viene inviato il codice di stato 404 Not Found	Valore atteso 404 Not Found	
DELETE api/v2/users/private						
1	Eliminare un utente dato un accessToken	accessToken	L'utente ha eseguito l'accesso e possiede un accessToken	Viene restituito il messaggio "Account eliminato con successo" e viene inviato il codice di stato 200 OK	Valore atteso 200 OK	

2	Eliminare un utente senza un accessToken	Nessuno	L'utente non ha eseguito l'accesso e non possiede un accessToken	Viene restituito il codice di stato 401 Unauthorized	Valore atteso 401 Unauthorized	
GET api/v2/admin/manageActivities						
1	Ottenere come admin con accessToken valido la lista di attività segnalate in ordine di segnalazioni con almeno un'attività segnalata	accessToken (admin)	L'admin ha eseguito l'accesso e possiede un accessToken ed esiste almeno un'attività segnalata presente	Viene restituito un array contenente tutte le attività segnalate in ordine di segnalazioni e il codice di stato 200 OK	Valore atteso 200 OK	
2	Ottenere come admin con accessToken	accessToken (admin)	L'admin ha eseguito l'accesso e possiede un accessToken e	Viene restituito il messaggio "Nessuna attività con segnalazioni" e il	Valore atteso 204 No Content	

	ken valido la lista di attività segnalate in ordine di segnalazi oni senza nessuna attività segnalata		non esiste nessuna attività segnalata presente	codice di stato 204 No Content		
3	Ottenere la lista di attività segnalate in ordine di segnalazi oni	Nessuno	Non è presente alcun accessToken nella richiesta di accesso alla risorsa	Viene restituito il codice di stato 401 Unauthorized	Valore atteso 401 Unauthor ized	
4	Ottenere come user con accessTo ken non valido la lista di attività segnalate in ordine di segnalazi oni	accessTo ken (user)	L'user ha eseguito l'accesso e possiede un accessToken non valido	Viene restituito il messaggio "Non sei autorizzato ad accedere a questa risorsa" e il codice di stato 403 Forbidden	Valore atteso 403 Forbidde n	

DELETE api/v2/admin/manageActivities/{id}						
1	Eliminare come admin con accessToken valido un'attività qualsiasi dato un idAttività valido	accessToken (admin), idAttività	L'admin ha eseguito l'accesso e possiede un accessToken ed esiste un'attività che corrisponde all'idAttività inserito	Viene restituito il messaggio "Attività eliminata con successo" e il codice di stato 200 OK	Valore atteso 200 OK	
2		Nessuno	Non è presente alcun accessToken nella richiesta di accesso alla risorsa	Viene restituito il codice di stato 401 Unauthorized	Valore atteso 401 Unauthorized	
3	Eliminare come user con accessToken non valido un'attività qualsiasi dato un	accessToken (user)	L'user ha eseguito l'accesso e possiede un accessToken non valido	Viene restituito il messaggio "Non sei autorizzato ad accedere a questa risorsa" e il codice di stato 403 Forbidden	Valore atteso 403 Forbidden	

	idAttività valido					
4	Eliminare come admin con accessToken valido un'attività qualsiasi dato un idAttività non valido	accessToken (admin), idAttività	L'admin ha eseguito l'accesso e possiede un accessToken e non esiste nessuna attività che corrisponde all'idAttività inserito	Viene restituito il messaggio "Attività non trovata" e il codice di stato 404 Not Found	Valore atteso 404 Not Found	
GET api/v2/admin/manageUsers/{query}						
1	Ottenere come admin con accessToken valido le informazioni dell'username corrispondente alla	accessToken (admin)	L'admin ha eseguito l'accesso e possiede un accessToken	Vengono restituiti tutti i dati dell'utente, meno la password, il cui username corrisponde alla stringa di ricerca e il codice di stato 200 OK	Valore atteso 200 OK	

	stringa di ricerca					
2	Ottenere come le informazioni dell'username corrispondente alla stringa di ricerca	Nessuno	Non è presente alcun accessToken nella richiesta di accesso alla risorsa	Viene restituito il codice di stato 401 Unauthorized	Valore atteso 401 Unauthorized	
3	Ottenere come user con accessToken non valido le informazioni dell'username corrispondente alla stringa di ricerca	accessToken (user)	L'user ha eseguito l'accesso e possiede un accessToken non valido	Viene restituito il messaggio "Non sei autorizzato ad accedere a questa risorsa" e il codice di stato 403 Forbidden	Valore atteso 403 Forbidden	
PUT api/v2/admin/manageUsers/{id}						

1	Promuovere come admin con accessToken valido un utente ad admin dato il suo userId valido	accessToken (admin), userId	L'admin ha eseguito l'accesso e possiede un accessToken ed esiste un utente associato all'userId inserito	Viene creato un nuovo admin con i dati dell'utente associato all'userId e viene restituito il messaggio "Utente promosso ad admin" e il codice di stato 200 OK	Valore atteso 200 OK	
2	Promuovere come admin con accessToken valido un utente che è già admin dato il suo userId valido	accessToken (admin), userId	L'admin ha eseguito l'accesso e possiede un accessToken ed esiste un admin associato all'userId inserito	Viene restituito il messaggio "Questo utente è già admin" e il codice di stato 400 Bad Request	Valore atteso 400 Bad Request	
3	Promuovere un utente ad admin	Nessuno	Non è presente alcun accessToken nella richiesta di accesso alla risorsa	Viene restituito il codice di stato 401 Unauthorized	Valore atteso 401 Unauthorized	

4	Promuovere come user con accessToken non valido un utente ad admin	accessToken (user)	L'user ha eseguito l'accesso e possiede un accessToken non valido	Viene restituito il messaggio "Non sei autorizzato ad accedere a questa risorsa" e il codice di stato 403 Forbidden	Valore atteso 403 Forbidden	
5	Promuovere come admin con accessToken valido un utente ad admin dato un userId non valido	accessToken (admin), userId	L'admin ha eseguito l'accesso e possiede un accessToken e non esiste un utente associato all'userId inserito o l'userId non è valido	Viene restituito il messaggio "Utente non trovato" ed il codice di errore 404 Not Found	Valore atteso 404 Not Found	
DELETE api/v2/admin/manageUsers/{id}						
1	Eliminare come admin con accessToken valido un utente	accessToken (admin), userId	L'admin ha eseguito l'accesso e possiede un accessToken ed esiste un utente non admin associato	Viene eliminato l'utente con userId corrispondente e viene restituito il messaggio "Utente eliminato con successo" e il codice di stato 200 OK	Valore atteso 200 OK	

	non admin dato un userId valido		all'userId inserito			
2	Eliminar e come admin con accessTo ken valido un utente admin dato un userId valido	accessTo ken (admin), userId (admin)	L'admin ha eseguito l'accesso e possiede un accessToken ed esiste un utente admin associato all'userId inserito	Viene inviato il messaggio "Non puoi eliminare un admin" e il codice di errore 400 Bad Request	Valore atteso 400 Bad Request	
3	Eliminar e un utente non admin	Nessuno	Non è presente alcun accessToken nella richiesta di accesso alla risorsa	Viene restituito il codice di stato 401 Unauthorized	Valore atteso 401 Unauthor ized	
4	Eliminar e come user con accessTo ken non valido un utente	accessTo ken (user)	L'user ha eseguito l'accesso e possiede un accessToken non valido	Viene restituito il messaggio "Non sei autorizzato ad accedere a questa risorsa" e il codice di stato 403 Forbidden	Valore atteso 403 Forbidde n	
5	Eliminar e come admin	accessTo ken	L'admin ha eseguito l'accesso e	Viene restituito il messaggio "Utente non trovato" e il	Valore atteso	

	con accessToken valido un utente non admin dato un userId non valido	(admin), userId	possiede un accessToken e non esiste un utente non admin associato all'userId inserito oppure l'userId non è valido	codice di stato 404 Not Found	404 Not Found	
GET api/v2/ad min/man ageUsers						
1	Ottenere come admin autenticat o con accessToken valido tutti gli utenti presenti nel sistema esclusi gli admin	accessTo ken (admin)	L'admin ha eseguito l'accesso e possiede un accessToken valido	Viene restituita la lista di utenti non admin presenti nel sistema e i relativi dati esclusa password ed il codice di stato 200 OK	Valore atteso 200 OK	
2	Ottenere tutti gli utenti presenti	Nessuno	Non è presente alcun accessToken nella richiesta di	Viene restituito il codice di stato 401 Unauthorized	Valore atteso 401	

	nel sistema esclusi gli admin		accesso alla risorsa		Unauthorized	
3	Ottenere come user autenticato con accessToken non valido tutti gli utenti presenti nel sistema esclusi gli admin	accessToken (user)	L'user ha eseguito l'accesso e possiede un accessToken non valido	Viene restituito il messaggio "Non sei autorizzato ad accedere a questa risorsa" e il codice di stato 403 Forbidden	Valore atteso 403 Forbidden	

6. Front-end

AroundYou è una web-app dinamica, minimal e semplice da utilizzare. Ci siamo concentrati principalmente sul rendere accattivante e facilitare l'utilizzo da parte dell'utente oltre a far sì che ogni parte del sito sia coesa con le altre e le funzionalità sviluppate siano efficienti.

Per la realizzazione del front-end abbiamo utilizzato un framework come vue.js, tailwind.css per la parte estetica e Leaflet per la mappa interattiva, strumenti indispensabili che hanno facilitato lo sviluppo."

6.1 Pagine di AroundYou

Il frontend si interfaccia in prima persona con il consumatore finale, il quale ha la possibilità di affacciarsi ad ogni funzionalità sviluppata e inserita come la gestione dell'inserimento e la modifica dei dati personali, la ricerca, la creazione e la cancellazione delle attività.

- La web-app è suddivisa in:

- **Landing page** : la pagina che accoglie l'utente appena nel sito. Permette all'utente di comprendere quale è il significato del sito oltre a scegliere se registrarsi o passare direttamente alla pagina principale
- **Pagina principale - attività sul territorio** : è la pagina comune a tutti gli utenti, autenticati e non, dove è possibile visionare le attività create da tutti gli utenti, filtrarle per visionare quelle che ci interessano maggiormente, muovere la mappa e, se si è autenticati, salvare o segnalare un'attività
- **Pagina creazione** : sezione nella quale è presente un form che permette la creazione di una nuova attività dopo l'inserimento completo dei dati
- **Pagina registrazione utente** : pagina con un form che permette a qualsiasi utente non autenticato la creazione di profilo per sbloccare le funzionalità aggiuntive
- **pagina di accesso** : dopo la creazione di un profilo, questa pagina permette all'utente di autenticarsi e visionare il sito con dei privilegi e delle funzionalità sbloccate
- **dashboard utente semplice** : pagina privata e diversa per ogni utente, nella quale è possibile osservare le azioni svolte nell'app e modificare i dati inseriti
 - **pagina principale** : in questa pagina si possono visionare tutte le attività create, sia quelle terminate che quelle che devono ancora svolgersi, e quest'ultime possono essere modificate o eliminate
 - **salvati** : questa è la sezione riguardante le attività a cui abbiamo deciso di partecipare
 - **impostazione** : sezione indispensabile per modificare o eliminare il proprio profilo
- **dashboard admin** : la dashboard admin è composta da una unica pagina che presenta però 3 moduli distinti: attività ordinate in ordine decrescente per numero di segnalazioni, lista di tutti gli utenti non admin che si possono promuovere, lista delle attività create

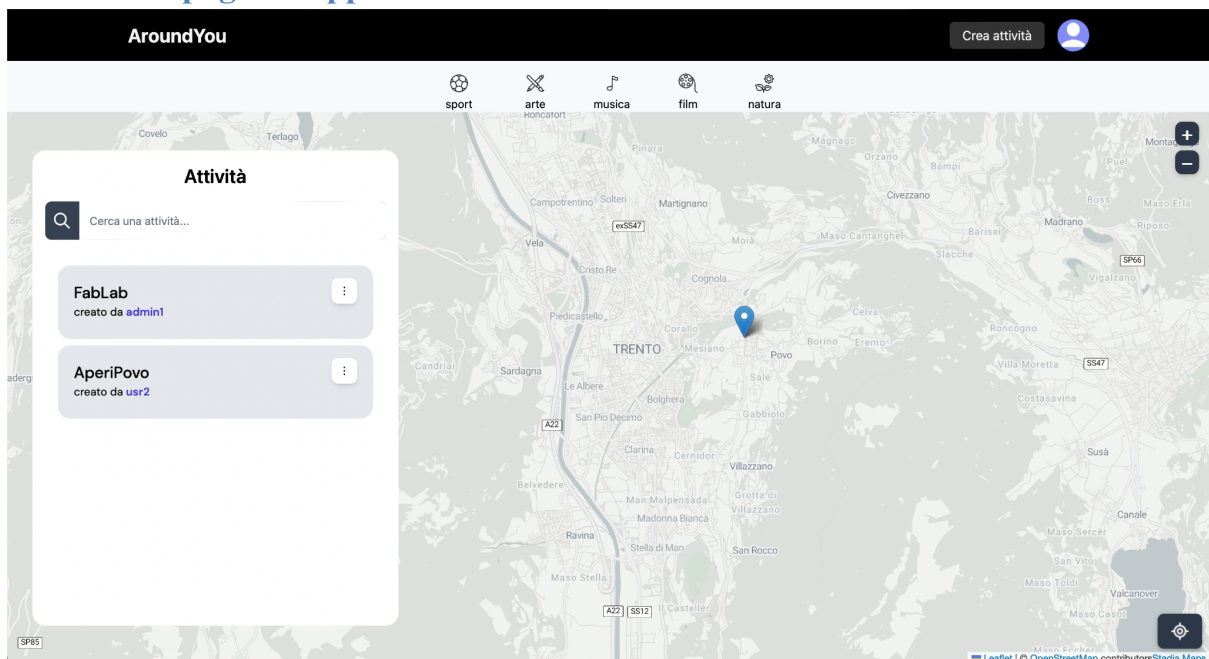
6.1.1 Landing page



La landing page è la prima pagina che l'utente osserva appena apre il sito. La frase racchiude il nostro obiettivo che ogni persona che sfrutterà il sito potrà fare.

Da questa sezione l'utente può decidere se registrarsi o andare a esplorare la pagina principale per capire di cosa l'app tratta.

6.1.2 Home page - mappa e attività



In questa pagina troviamo le principali funzionalità del sito: la mappa nella quale si può cliccare sui vari ping, la bacheca di liste le quali se cliccate aprono i banner e i topic maggiormente usati che filtrano direttamente le attività.

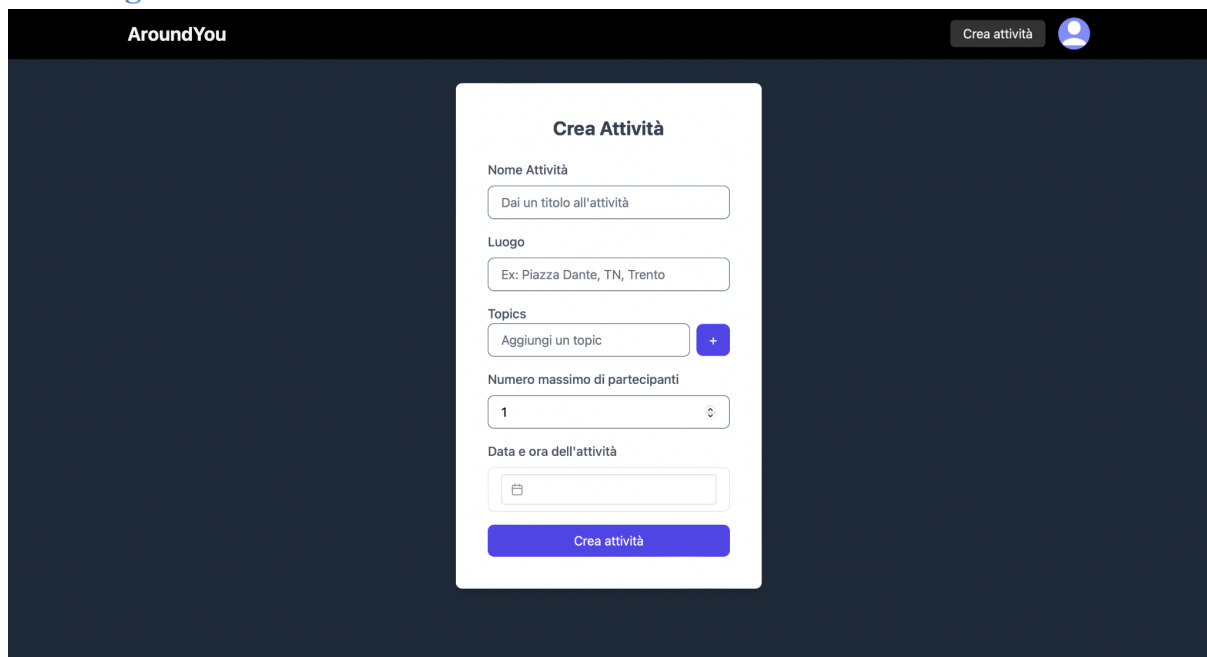
Il bottone in basso a destra della mappa riposiziona la mappa su Trento a una corretta distanza

Ogni attività se viene cliccata, fa sì che si apra un banner con tutte le informazioni e i bottoni di partecipazione e segnalazione

Il tasto “Crea attività” permette all’utente di navigare di andare sul form di creazione se autenticato.

Se viene cliccato l’omino invece si apre un banner per loggarsi o registrarsi, oppure se l’accesso è avvenuto di andare sulla dashboard o di fare il logout

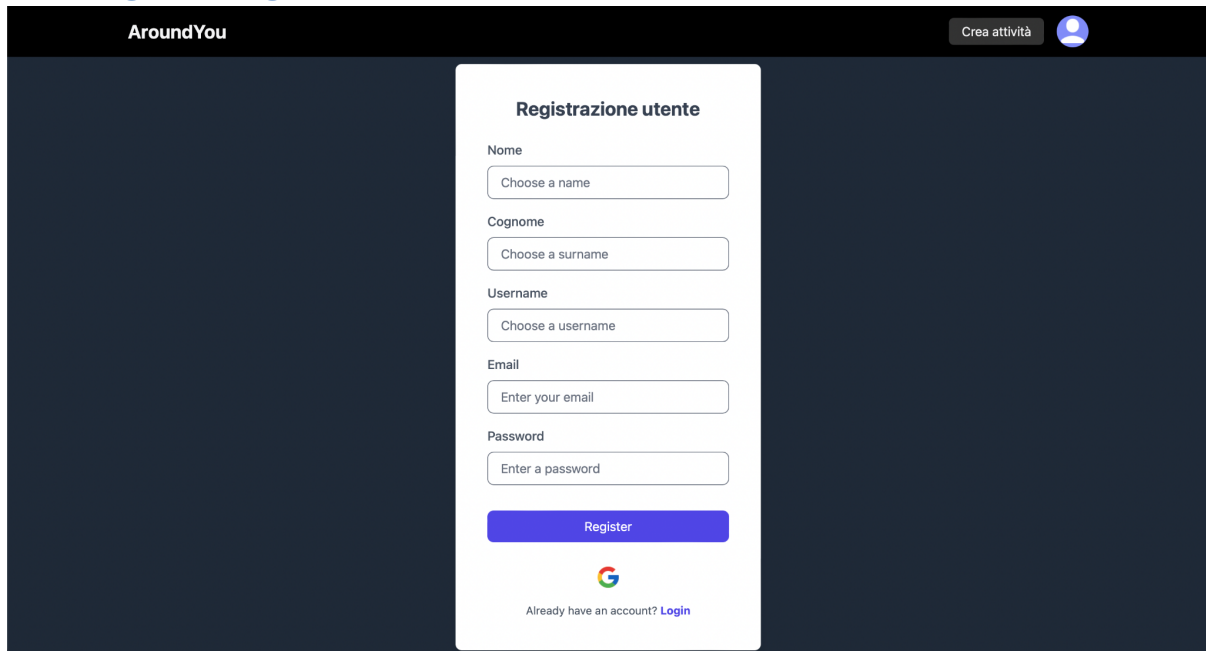
6.1.3 Pagina creazione nuova attività



The screenshot shows the 'Crea Attività' (Create Activity) form within the 'AroundYou' application. The form is centered on a dark blue background. At the top of the form, the title 'Crea Attività' is displayed. Below the title, there are several input fields: 'Nome Attività' with a placeholder 'Dai un titolo all'attività', 'Luogo' with a placeholder 'Ex: Piazza Dante, TN, Trento', 'Topics' with a placeholder 'Aggiungi un topic' and a plus button, 'Numero massimo di partecipanti' with a numeric input field set to '1', and 'Data e ora dell'attività' with a date and time picker. At the bottom of the form is a large blue button labeled 'Crea attività'. In the top right corner of the application header, there is a 'Crea attività' button and a user profile icon.

L’utente autenticato per mezzo di questo form, i cui campi sono tutti obbligatori, può registrare una nuova attività, che verrà inserita nel sito

6.1.4 Pagina di registrazione utente



Un utente non autenticato può creare un nuovo profilo inserendo i suoi dati sensibili e quelli utili ai successivi accessi:

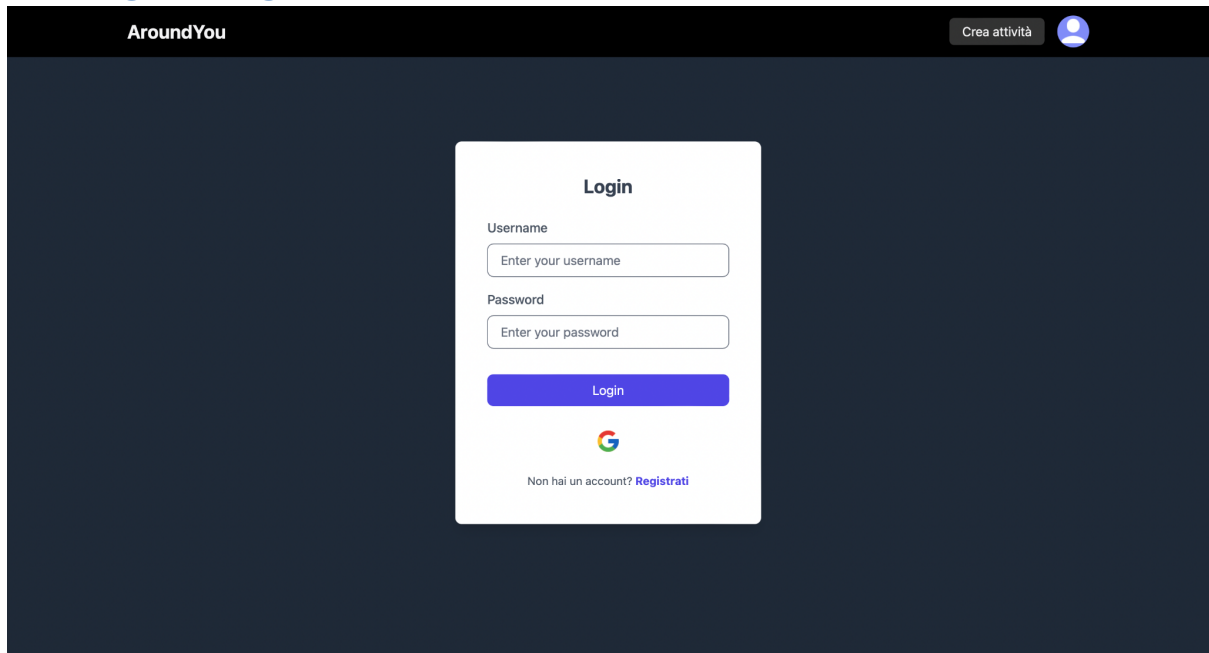
- Nome
- Cognome
- Username (utile per l'accesso e univoco)
- Email
- Password
- Conferma password

Se un utente non vuole registrarsi in modo manuale può decidere di registrarsi per mezzo di un account Google preesistente

Se invece è già registrato e vuole eseguire l'accesso può premere per poi essere ridirezionato sul login in blu.

Un admin non può crearsi in modo autonomo un profilo ma può crearne uno normale ed essere promosso da un admin, o farsi inserire manualmente nel database

6.1.5 Pagina di login



Un utente quindi se vuole sbloccare le funzionalità aggiuntive deve navigare nel sito dopo aver eseguito l'accesso.

Se un utente si è registrato manualmente, per poter accedere deve inserire il proprio username inserito in fase di registrazione e la propria password.

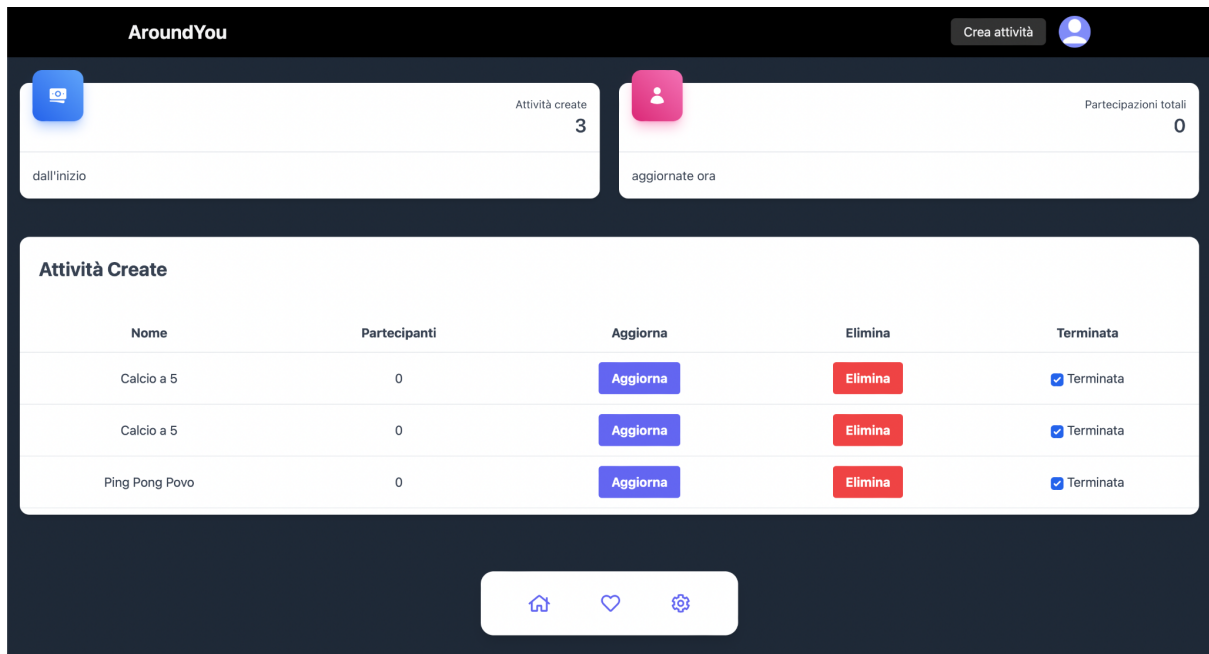
Nel caso invece la creazione del profilo sia avvenuta per mezzo di un account Google, può decidere di utilizzare quella modalità.

Se invece si è ritrovato nella pagina ma effettivamente non ha ancora un profilo, può navigare fino al form di registrazione premendo su registrati in blu

6.1.6 Dashboard utente semplice

La dashboard quindi l'area riservata di ogni utente è raggiungibile dopo aver eseguito il login. Se un utente vuole entrare nella sua area privata deve premere sull'icona in alto a destra presente in ogni pagina, la quale è dinamica, ovvero cambia nel momento in cui un utente ha eseguito l'accesso.

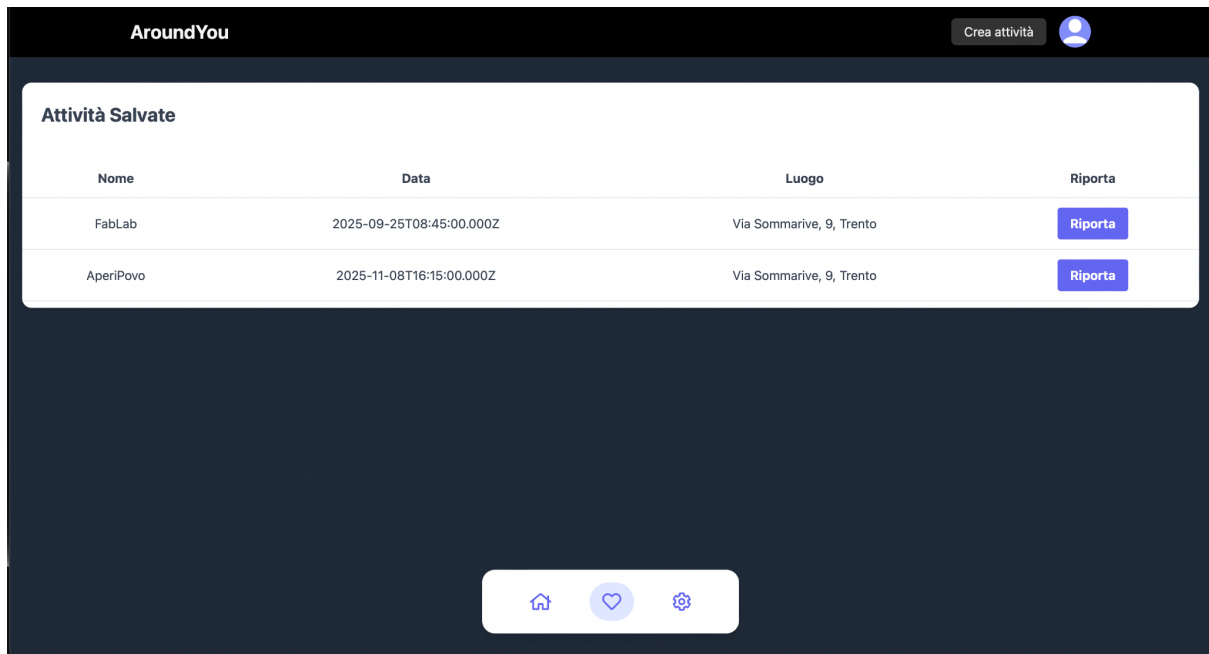
Il banner che si aprirà quindi presenta il bottone dashboard o logout.



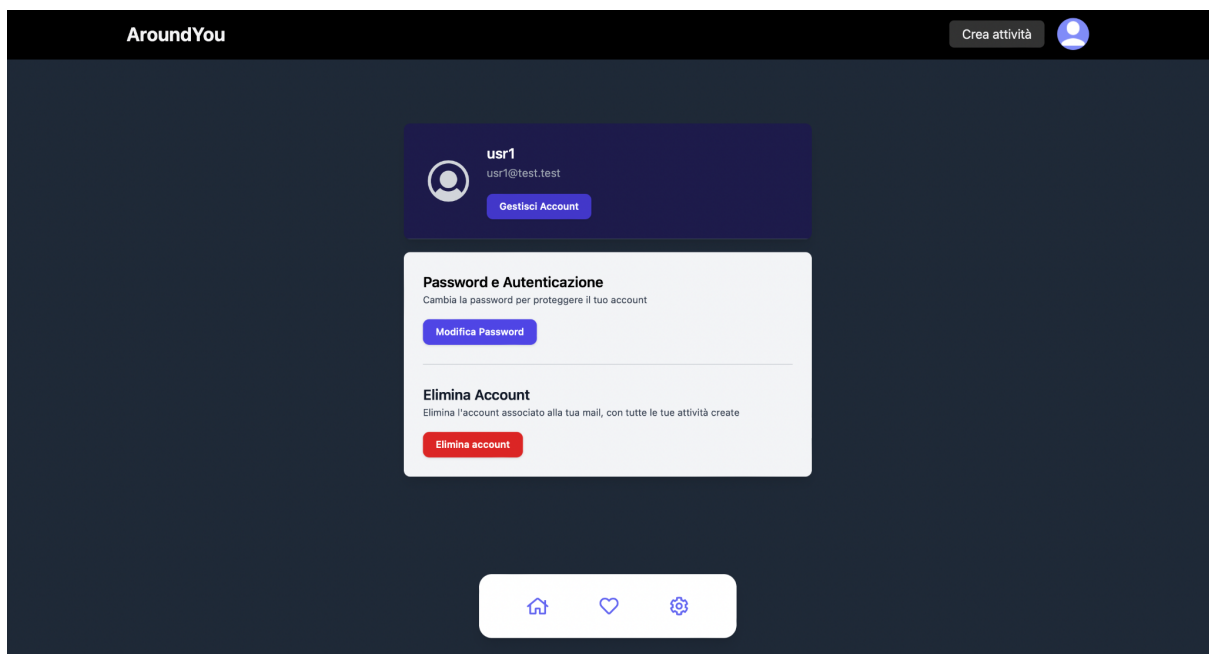
Prima pagina dell'area personale dell'utente semplice. Qui l'utente può visionare tutte le sue attività create e terminate. In alto troviamo due moduli che evidenziano tutte le attività create dall'inizio e tutte le partecipazioni che vengono aggiornate dinamicamente

Se le attività sono ancora in corso, l'utente le può modificare premendo sul tasto modifica, che aprirà un banner con un form di modifica.

Se invece l'attività non potrà più svolgersi, il creatore può eliminarla premendo sul tasto elimina



La seconda pagina dell'area personale invece mostra tutte le attività alle quali un utente ha deciso di partecipare. Vengono mostrati i dati principali e a lato di ogni attività è presente un bottone Riporta che permette all'utente, in caso di dubbi sulla legalità e veridicità dell'attività, di riportarla.



L'ultima pagina è quella delle impostazioni. Un utente può modificare i propri dati di accesso come la mail, lo username o la password e può cancellare il proprio profilo con tutte le attività annesse.

6.1.7 Dashboard admin

The screenshot shows the 'AroundYou' admin dashboard. At the top, there's a header with the 'AroundYou' logo and a 'Crea attività' button next to a user profile icon. The dashboard is divided into three main sections:

- Attività Segnalate:** A list of activities. The first entry is 'AperiPovo' with a timestamp '2025-11-08T16:15:00.000Z' and a user 'usr2'. There is an 'Elimina' button next to it.
- Cerca utente...:** A search bar and a table of users. The table has columns 'Nome Utente' and 'Azione'. It lists 'usr1' and 'usr2', each with 'Promuovi' and 'Elimina' buttons.
- Attività Create:** A table of activities. It has columns 'Nome', 'Partecipanti', 'Aggiorna', 'Elimina', and 'Terminata'. The first entry is 'FabLab' with 1 participant. There are 'Modifica' and 'Elimina' buttons, and a checkbox for 'In corso'.

La dashboard admin invece è composta da tre moduli: il primo riguarda una lista con tutte le attività segnalate e ordinate per numero di segnalazione; il secondo è una lista di utenti, i quali possono essere cercati, promossi e ai quali un admin può eliminare il profilo; infine troviamo la lista di attività create dall' admin specifico e si possono svolgere le stesse funzionalità di un utente classico sulle proprie attività

7. Deployment

Il frontend e backend sono stati distribuiti per mezzo del sito [Render.com](https://render.com) e ospitati assieme. È possibile provare il sito seguendo il link <https://aroundyou-puv3.onrender.com>.

La configurazione CI/CD, che si basa sulle GitHub Actions, esegue automaticamente il deploy del branch Main, e riesegue il deploy ogni qual volta si presenti un commit nuovo su quel branch.

Per poter testare il sito si possono usare due profili:

- Utente semplice:
 - Username : FabioB89
 - Password : FabioBelli_

- Admin:
 - Username : Admin
 - Password : AdminComune_

Si possono registrare nuovi utenti tranne l'admin che non può fare la registrazione, ma può promuovere gli utenti